Design and Analysis of Algorithms

SY AIML

2025-26

SEM - II

Unit-I **05 Hrs.**

Introduction: Analysis of control statements and loops, solving recurrence relations using tree, substitution, master method, analysis of quick sort and merge sort Problem Solving using divide and conquer algorithm- Max-Min problem, Strassen's Matrix Multiplication

Unit-II **06 Hrs.**

Greedy Method: Introduction, Problem solving using- fractional knapsack problem, activity se lection problem, job sequencing with deadline, Graph: Minimum Spanning trees(Kruskal's algorithm (Use find and union concept, Prim's algorithm), Single source shortest path (Dijkstra's algorithm), coin change problem.

Unit-III **08 Hrs.**

Dynamic Programming: Introduction, principle of optimality, Components of dynamic programming, characteristics of dynamic programming, Fibonacci problem, Coin Changing problem, 0/1 knapsack (table method), All pairs shortest paths (Floyd Warshall Algorithm), Single source shortest path (Bellman-Ford Algorithm), Matrix Chain Multiplication, Travelling salesperson problem, Longest Common Subsequence (LCS), Analysis of all Algorithms.

Unit-IV Backtracking and Branch-and-Bound: **04 Hrs.**

Basics of backtracking, N–queen problem, Sum of subsets, Graph coloring, Analysis of all Algorithms. Branch-and-Bound: Introduction, Types of BB and its properties, Fifteen Puzzle problem

Unit-V **03 Hrs.**

String Matching Algorithms: The naive string-matching algorithm, The Rabin Karp algorithm, String matching with finite automata, The Knuth Morris Pratt algorithm

Text Books:

- S. Sridhar, “Design and Analysis of Algorithms”, 1st Edition, Oxford Education, 2018.
- Goodrich M T, “Design and Analysis of Algorithms”, Wiley, New Delhi, 2021.
- Ellis Horowitz, Sartaj Sahni, S. Rajsekaran, “Fundamentals of computer algorithms”, University Press.

Reference Books:

- **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, "Introduction to Algorithms", 4th Edition, The MIT Press, 2022.**
- Sanjoy Dasgupta, Christos Papadimitriou, Umesh Vazirani, "Algorithms", Tata McGraw-Hill Edition.
- S. K. Basu, "Design Methods and Analysis of Algorithm", PHI.
- John Kleinberg, Eva Tardos, "Algorithm Design", Pearson.
- Michael T. Goodrich, Roberto Tamassia, "Algorithm Design", Wiley Publication.

- Evaluation Scheme:
- Theory :
- Continuous Assessment (A):

  Subject teacher will declare Teacher Assessment criteria at the start of semester.
- Continuous Assessment (B):

  1. Two term tests of 15 marks each will be conducted during the semester.

  2. Average of the marks scored in both the tests will be considered for final grading.
- End Semester Examination (C):

  1. Question paper based on the entire syllabus, summing up to 60 marks.

  2. Total duration allotted for writing the paper is 2 hrs

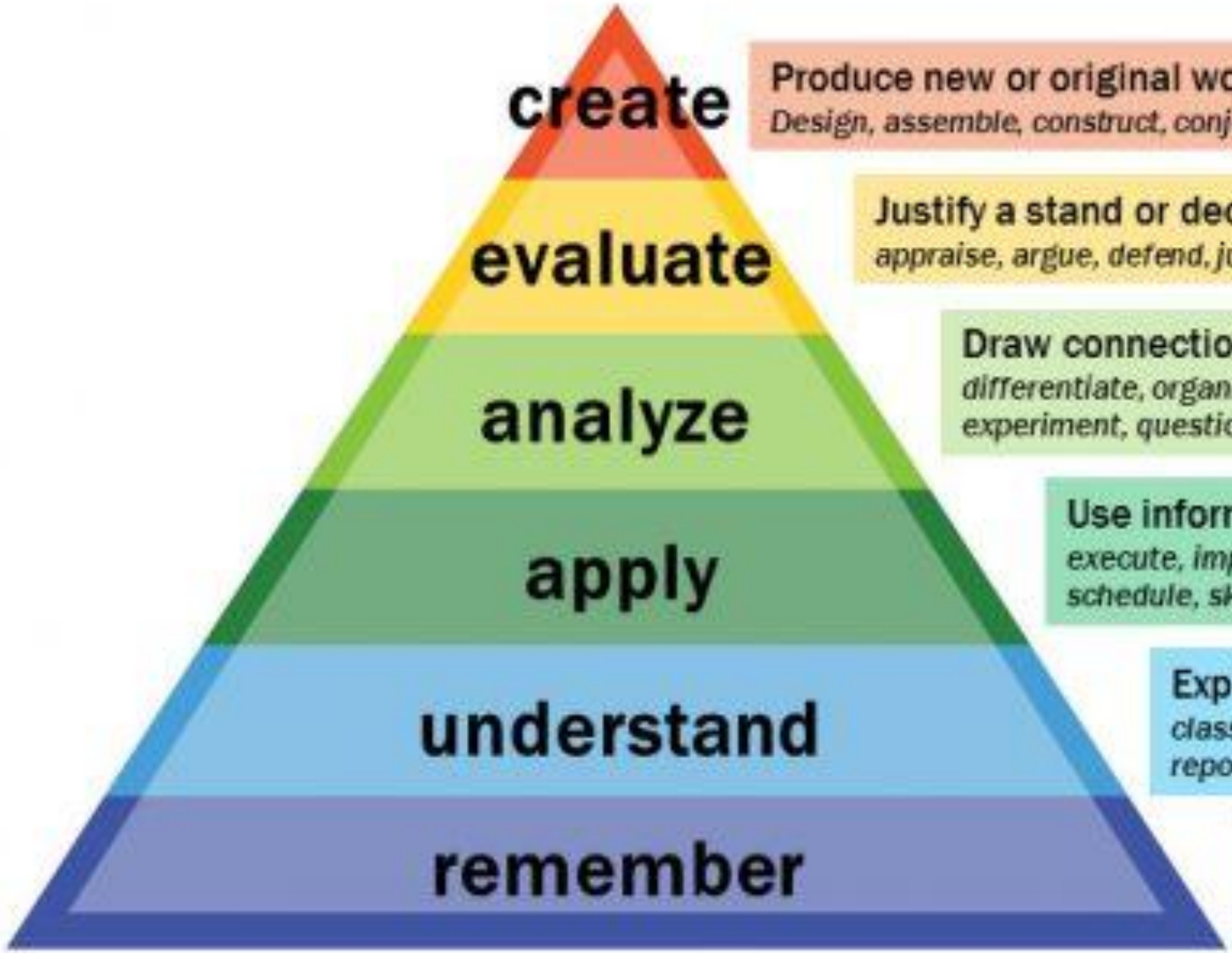
- **Evaluation Scheme:** **Laboratory**
- Continuous Assessment (TA):
- Laboratory work will be based on RCP23ACPC402 with **minimum 08** experiments to be incorporated.
- The distribution of marks for term work shall be as follows:

1. Performance in Experiments: 05 Marks
2. Journal Submission: 05 Marks
3. Viva-voce: 05 Marks
4. Subject Specific Lab Assignment/Case Study: 10 Marks

- End Semester Examination (ESE):

Oral & Practical examination will be based on the entire syllabus including, the practical's performed during laboratory sessions.

Bloom's Taxonomy



Produce new or original work
Design, assemble, construct, conjecture, develop, formulate, author, investigate

Justify a stand or decision
appraise, argue, defend, judge, select, support, value, critique, weigh

Draw connections among ideas
differentiate, organize, relate, compare, contrast, distinguish, examine, experiment, question, test

Use information in new situations
execute, implement, solve, use, demonstrate, interpret, operate, schedule, sketch

Explain ideas or concepts
classify, describe, discuss, explain, identify, locate, recognize, report, select, translate

Recall facts and basic concepts
define, duplicate, list, memorize, repeat, state

R. C. PATEL INSTITUTE OF TECHNOLOGY

(An Autonomous Institute)

Unit-I Introduction 05 Hrs.

- Introduction to Asymptotic Analysis
- Analysis of control statements and loops
- solving recurrence relations using tree method
- solving recurrence relations using substitution method
- solving recurrence relations using master method
- analysis of quick sort
- analysis of merge sort
- Problem Solving using divide and conquer algorithm - Max-Min problem
- Strassen’s Matrix Multiplication.

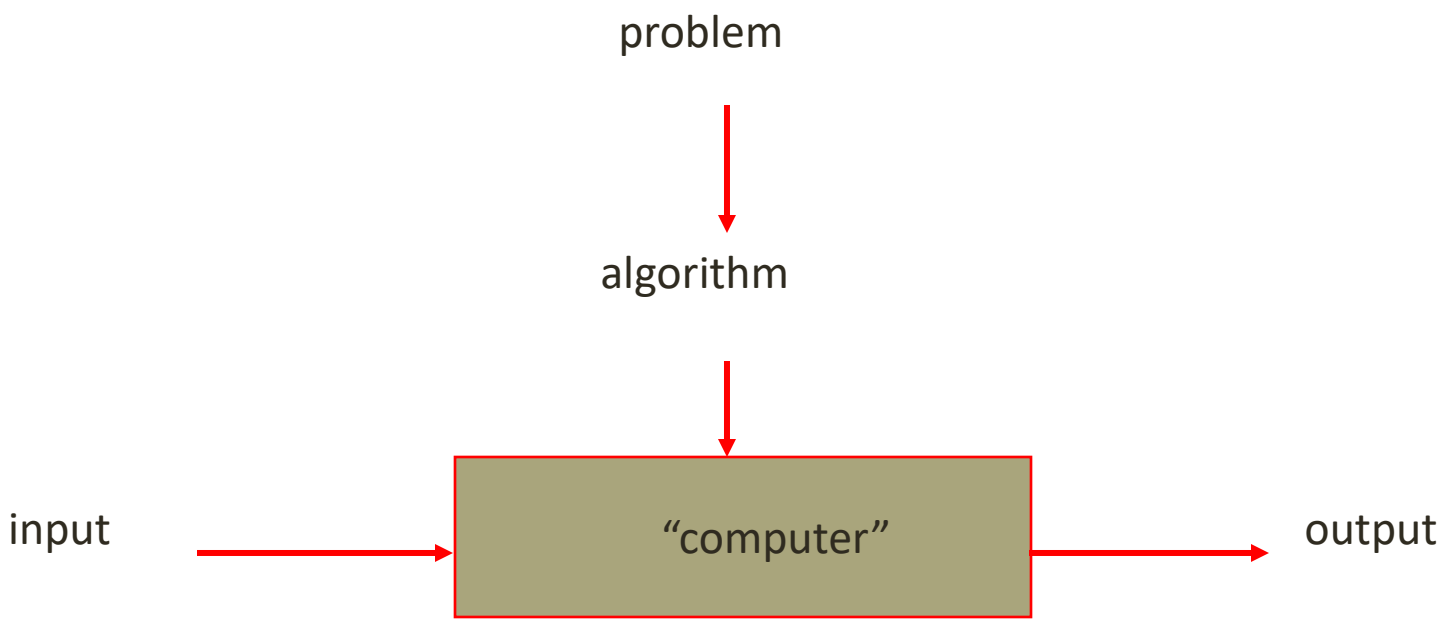
Algorithm

- An **algorithm** is a clear step-by-step method used to solve a problem, especially in mathematics or computing.
- The word *algorithm* comes from the name of a Persian mathematician **Al-Khwarizmi**.

- Algorithms can be of different types, such as:
  - **Recursive** (a function calls itself)
  - **Serial** (steps done one after another)
  - **Parallel or Distributed** (multiple steps done at the same time)

- We can write an algorithm in many forms:
  - Normal (English) language
  - Programming language
  - Pseudocode (code-like steps)
  - Flowchart diagrams

Concept of Algorithm

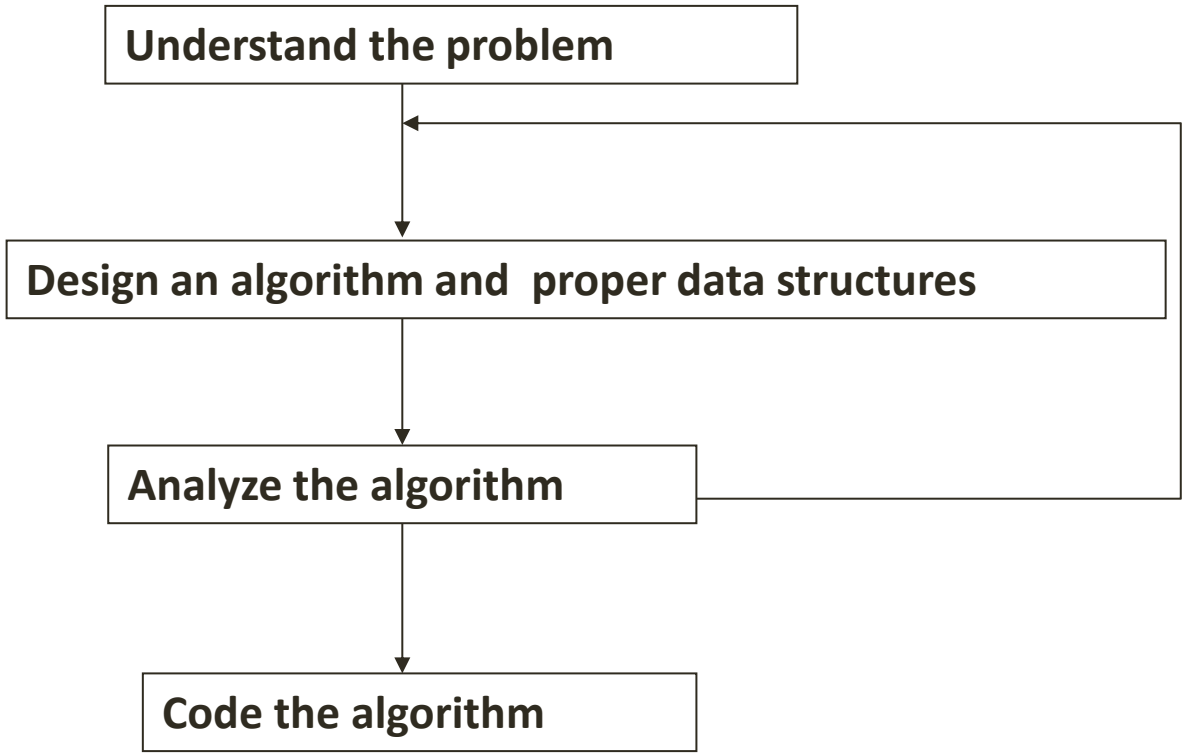


Algorithmic solution

Definitions of Algorithm

- A mathematical relation between an observed quantity and a variable used in a step-by-step mathematical process to calculate a quantity.
- **Algorithm is any well defined computational procedure that takes some value or set of values as input and produces some value or set of values as output.**
- A procedure for solving a mathematical problem in a finite number of steps that frequently involves repetition of an operation; broadly : a step-by-step procedure for solving a problem or accomplishing some end (Webster's Dictionary).

Algorithmic Problem Solving



Steps in solving a problem using algorithms:

1. **Understand the computer's ability**
   Know what the computer can do and its limits (memory, speed, etc.).

2. Choose exact or approximate solution
Decide whether you need a perfectly accurate answer or a close enough answer.

3. Select proper data structure
Choose how to store data (array, list, stack, queue, tree, etc.).

4. Design the algorithm
Plan the step-by-step procedure to solve the problem.

5. Analyze the algorithm
Check how fast it works and how much memory it uses.

6. Coding (Implementation)
Convert the algorithm into a program using a programming language.

Basic Issues Related to Algorithms

- **How to design algorithms**

  How to **create step-by-step methods** to solve a problem.

- **How to express algorithms**

  How to **write or show the algorithm** so others can understand it.

- **Proving correctness**

  Showing that the algorithm **always gives the correct answer**.

- **Efficiency**

  How **fast** and **memory-saving** the algorithm is.

  - Theoretical analysis
  - Experimental analysis

- **Optimality / Best Solution**

  Finding the **best possible solution** among all algorithms.

Analysis of Algorithms

- How good is the algorithm?
  - Correctness
  - Time efficiency
  - Space efficiency

What is an algorithm? (Properties)

1. **Unambiguous**: Every step must be clearly defined.
2. **Input**: Can have zero or more inputs.
3. **Output**: Must produce at least one output.
4. **Finiteness**: Should terminate after a finite number of steps.
5. **Effectiveness**: Each step must be basic enough to be carried out.
6. **Independence**: Should be independent of any programming language

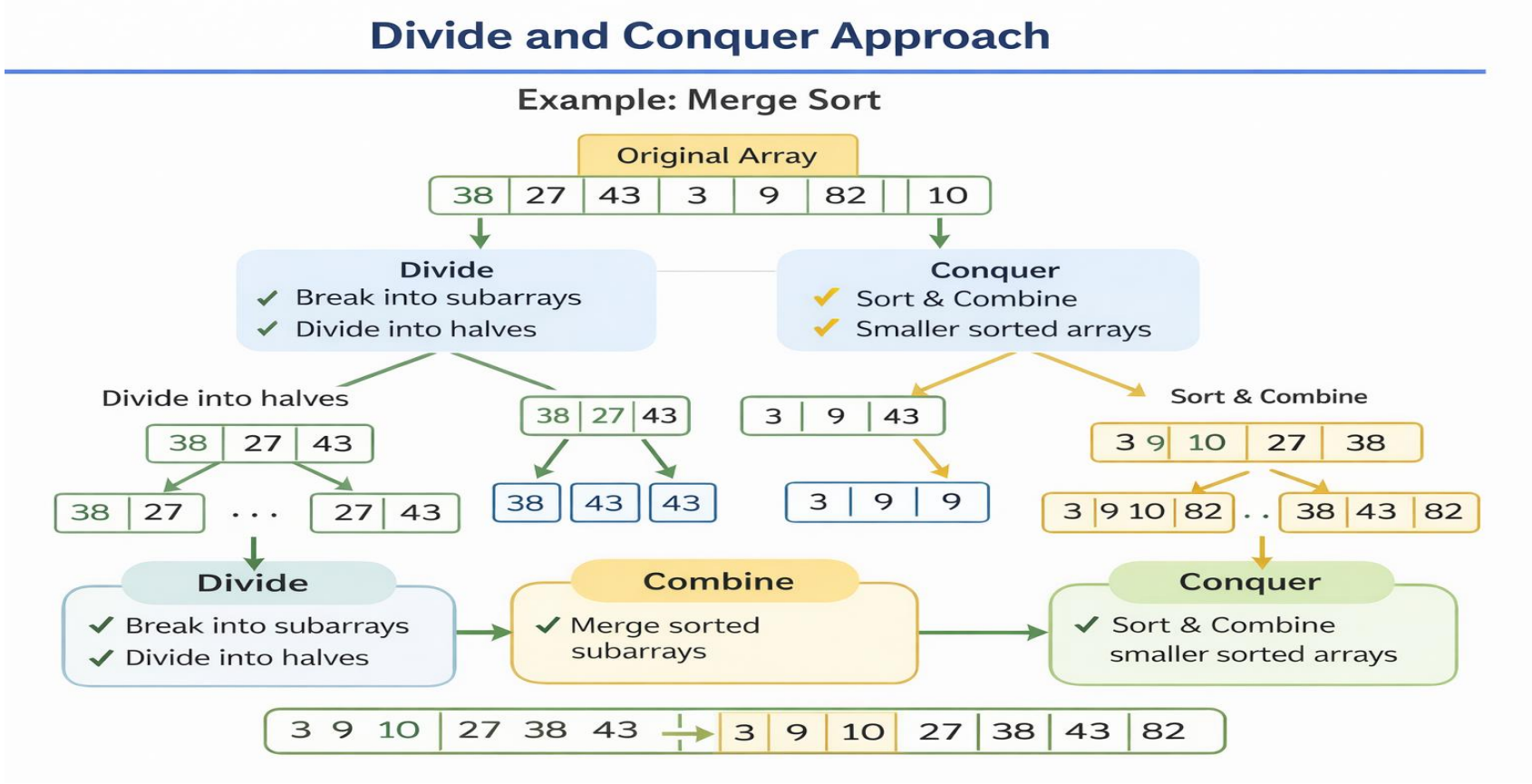
Algorithm Designing Methods / Design Techniques

1. Incremental Approach
2. Divide and Conquer
3. Dynamic Programming
4. Greedy Approach
5. Backtracking
6. Linear Programming
7. Branch and Bound method
8. Randomized Algorithm

1. Divide and Conquer Approach:

It is a top-down approach. The algorithms which follow the divide & conquer techniques involve three steps:

- Divide the original problem into a set of subproblems.
- Solve every subproblem individually, recursively.
- Combine the solution of the subproblems (top level) into a solution of the whole original problem.



2. Greedy Technique:

The **Greedy algorithm** is used to solve problems where we want the **best result** — either:

- Maximum profit
- Minimum cost
- Minimum time
- Shortest distance

These are called **optimization problems**.

Example— Coin Change Problem

You want to make **₹18** using coins:

₹10, ₹5, ₹2, ₹1

Greedy Approach

Always pick largest coin first.

Take 10 → Remaining = 8

Take 5 → Remaining = 3

Take 2 → Remaining = 1

Take 1 → Remaining = 0

Total coins used = 4 (Good solution)

3. Dynamic Programming:

Dynamic Programming (DP) is a technique used to solve complex problems by:

Solving **small sub-problems first**

Saving their answers

Using them again to build the final big answer

So instead of solving the same problem again and again,

we **remember previous results**.

- **Example — Fibonacci Numbers**

We want to find:

F(n)=F(n−1)+F(n−2)……

0, 1, 1, 2, 3, 5, 8, 13, 21 …

4. Branch and Bound :

Branch and Bound is a method used to find the **best possible solution** from many choices.

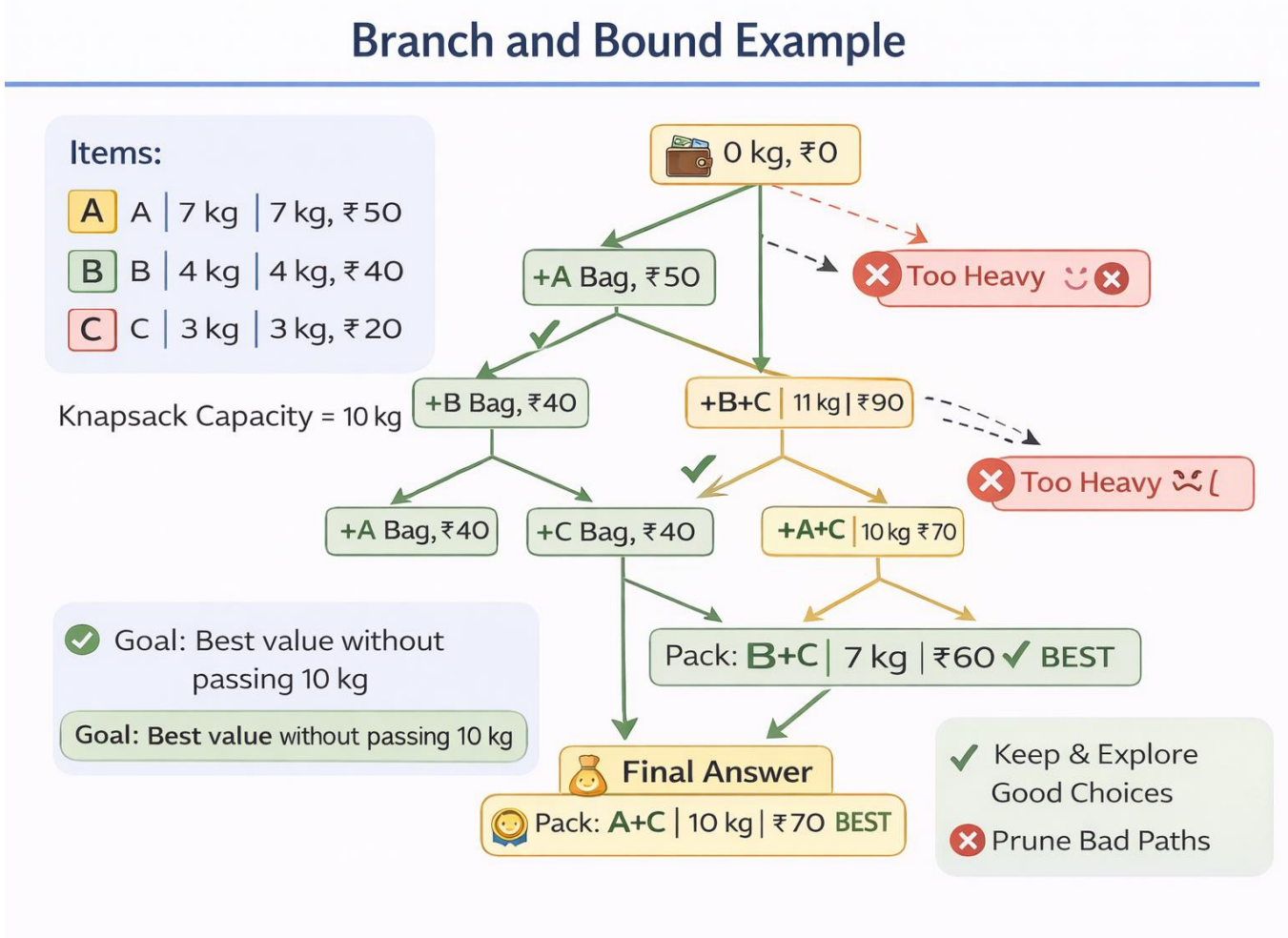
It works like a tree:

Branch → Try different possible choices

Bound → Stop exploring a choice if it is alreadv worse than the best answer

In short:

Try all good paths, skip bad paths early



5. Randomized Algorithms:

A **Randomized Algorithm** is an algorithm that uses **random numbers (luck/chance)** while solving a problem.
So every time you run it →
the steps may be slightly different →
but the answer is still correct (or almost correct).

6. Backtracking Algorithm: A backtracking algorithm is a method of solving problems by **trying all possible options one by one**. If one option does not lead to the correct answer, it **goes back (backtracks)** to the previous step and **tries a different option**.

Introduction to Analysis of Algorithm

- Some people enjoy studying algorithms because it is **fun and challenging**.
- They like to **predict how fast and efficiently a program will work**.
- Computer science attracts people who like **making things faster and better**.
- Algorithm analysis means **checking how much time and memory a program needs**.

Algorithm Analysis

- Analyzing means **predicting resources** that algorithms requires , i.e.
- Space complexity
  - How much space is required
- Time complexity
  - How much time does it take to run the algorithm

Space Complexity

- **Space Complexity** means **how much memory an algorithm needs to run completely**.

- Sometimes, a program uses **more memory than the system has**, which can cause errors like **core dumps**. The most common reason for this is **memory leaks**, where memory is not released properly.

- Some algorithms work **faster and better if all the data is loaded into memory**, but this is not always possible. So, we must also **consider the memory limits of the system**.

Time Complexity

- Time complexity is **often more important than space complexity**.
- Memory is increasing, but **time is still limited**.
- Even very fast computers need **a long time for complex problems**.
- Therefore, **algorithm running time is a critical issue**.

Basic Matrix Multiplication

Suppose we want to multiply two matrices of size N x N: for example A x $B = C$.

$$\begin{vmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{vmatrix} = \begin{vmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{vmatrix} \begin{vmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{vmatrix}$$

$\mathbf{C_{11} = a_{11}b_{11} + a_{12}b_{21}}$

$\mathbf{C_{12} = a_{11}b_{12} + a_{12}b_{22}}$

$\mathbf{C_{21} = a_{21}b_{11} + a_{22}b_{21}}$

$\mathbf{C_{22} = a_{21}b_{12} + a_{22}b_{22}}$

2x2 matrix multiplication can be accomplished in 8 multiplications.

$\mathbf{(2^{\log_2 8} = 2^3)}$

Divide and Conquer Matrix Multiply

A × B = R

A_0	A_1
A_2	A_3

×

B_0	B_1
B_2	B_3

=

$\mathbf{A_0 \times B_0 + A_1 \times B_2}$	$\mathbf{A_0 \times B_1 + A_1 \times B_3}$
$\mathbf{A_2 \times B_0 + A_3 \times B_2}$	$\mathbf{A_2 \times B_1 + A_3 \times B_3}$

- Divide matrices into sub-matrices: A_0 , A_1, A_2 etc
- Use blocked matrix multiply equations
- Recursively multiply sub-matrices

Strassens's Matrix Multiplication

$$\begin{vmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{vmatrix} = \begin{vmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{vmatrix} \begin{vmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{vmatrix}$$

$P_1 = (A_{11} + A_{22})(B_{11} + B_{22})$

$P_2 = (A_{21} + A_{22}) * B_{11}$

$P_3 = A_{11} * (B_{12} - B_{22})$

$P_4 = A_{22} * (B_{21} - B_{11})$

$P_5 = (A_{11} + A_{12}) * B_{22}$

$P_6 = (A_{21} - A_{11}) * (B_{11} + B_{12})$

$P_7 = (A_{12} - A_{22}) * (B_{21} + B_{22})$

$C_{11} = P_1 + P_4 - P_5 + P_7$

$C_{12} = P_3 + P_5$

$C_{21} = P_2 + P_4$

$C_{22} = P_1 + P_3 - P_2 + P_6$

No. of multiplications: M(n) = 7

No. of additions: A(n) = 18

$2^{\log_2 7} = 2^{2.807} = \Theta(n^{2.807})$

Strassen's matrix multiplication

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} * \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

$$= \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{bmatrix}$$

$M_1=(A_{00}+A_{11})*(B_{00}+B_{11})$
$M_2=(A_{10}+A_{11})*B_{00}$
$M_3=A_{00}*(B_{01}-B_{11})$
$M_4=A_{11}*(B_{10}-B_{00})$
$M_5=(A_{00}+A_{01})*B_{11}$
$M_6=(A_{10}-A_{00})*(B_{00}+B_{01})$
$M_7=(A_{01}-A_{11})*(B_{10}+B_{11})$

recurrence relations:

multiplication: M(n) = 7

addition: A(n) = 18

Strassen's method for matrix multiplication

Strassen's algorithm

- Define seven matrices of size (n/2)x(n/2) :

$$\underbrace{\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}}_{A} \cdot \underbrace{\begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}}_{B} = \underbrace{\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}}_{C}$$

$$\begin{aligned}
M_1 &= (A_{11} + A_{22}) \cdot (B_{11} + B_{22}) \\
M_2 &= (A_{21} + A_{22}) \cdot B_{11} \\
M_3 &= A_{11} \cdot (B_{12} - B_{22}) \\
M_4 &= A_{22} \cdot (B_{21} - B_{11}) \\
M_5 &= (A_{11} + A_{12}) B_{22} \\
M_6 &= (A_{21} - A_{11})(B_{11} - B_{12}) \\
M_7 &= (A_{12} - A_{22})(B_{21} - B_{22})
\end{aligned}$$

- The four (n/2)x(n/2) matrices C_{ij} can be defined in terms of M1,…,M7:

$$\begin{aligned}
C_{11} &= M_1 + M_4 - M_5 + M_7 \\
C_{12} &= M_3 + M_5 \\
C_{21} &= M_2 + M_4 \\
C_{22} &= M_1 - M_2 + M_3 + M_6
\end{aligned}$$

Strassen's matrix multiplication

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} * \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

$$= \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{bmatrix}$$

$M_1=(A_{00}+A_{11})*(B_{00}+B_{11})$
$M_2=(A_{10}+A_{11})*B_{00}$
$M_3=A_{00}*(B_{01}-B_{11})$
$M_4=A_{11}*(B_{10}-B_{00})$
$M_5=(A_{00}+A_{01})*B_{11}$
$M_6=(A_{10}-A_{00})*(B_{00}+B_{01})$
$M_7=(A_{01}-A_{11})*(B_{10}+B_{11})$

$$T(n) = \begin{cases} b & n =< 2 \\ 7T(n/2) + an^2 & n > 2 \end{cases}$$

$M(n) \approx \Theta(n^{2.807})$

$A(n) \approx \Theta(n^{2.807})$

Strassens's Matrix Multiplication

- Strassen showed that 2x2 matrix multiplication can be accomplished in 7 multiplication and 18 additions or subtractions. .($2^{\log_2 7} = 2^{2.807}$)
- This reduce can be done by Divide and Conquer Approach.
- Time Complexity $T(N) = 7T(N/2) + O(N^2)$

- **Lower bound**: a value that is less than or equal to every element of a set of data.
- **Upper bound**: a value that is greater than or equal to every element of a set of data.
- Example: in { 3, 5, 11, 20, 22 }

  3 is a lower bound, and

  22 is an upper bound.

Measurement of Complexity of an Algorithm

Time Complexity there are three cases to analyse an algorithm:

1. Worst Case Analysis (Mostly used)

- we calculate the **upper bound** on the running time of an algorithm.
- causes a **maximum number of operations to be executed**.
- For Linear Search, the worst case happens when the element to be searched (x) is **not present in the array**. When x is not present, the search() function compares it with all the elements of arr[] one by one. Therefore, the worst-case time complexity of the linear search would be **O(n)**.

2. Best Case Analysis (Very Rarely used)

- we calculate the **lower bound** on the running time of an algorithm.
- We must know the case that causes a **minimum number of operations to be executed**.
- In the linear search problem, the best case occurs when x is present at the first location. The number of operations in the best case is constant (not dependent on n). So time complexity in the best case would be $\Omega(1)$

3. Average Case Analysis (Rarely used)

- we take all possible inputs and calculate the computing time for all of the inputs.
- Sum all the calculated values and divide the sum by the total number of inputs.
- We must know (or predict) the distribution of cases.

Asymptotic Notation

- Big-Oh **O**
- Small-Oh **o**
- Big Omega **Ω**
- Small-Omega **ω**
- Theta Notations **Θ**

- **mathematical tool**
- **used to describe the running time of an algorithm** - how much time an algorithm takes with a given input, n.
- three different notations:

big O, big Theta (Θ), and big Omega (Ω).

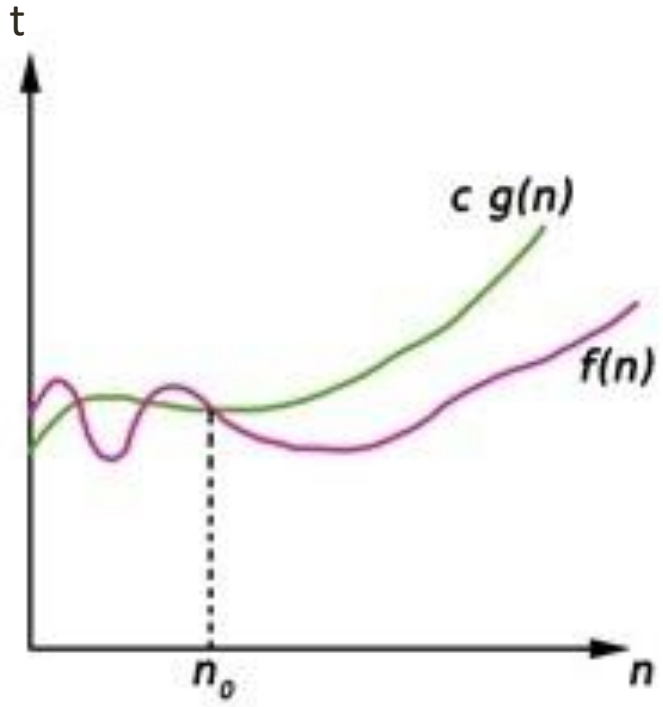
Popular Notations in Complexity Analysis of Algorithms

1. Big-O Notation

- We define an algorithm's **worst-case** time complexity by using the Big-O notation, **which determines the set of functions grows slower than or at the same rate as the expression.**
- It explains the **maximum amount of time** an algorithm requires to consider all input values.

Big-Oh (O) notation (worst case)

- gives an **upper bound** for a function f(n) to within a constant factor. (n is the size of input)
- f(n) = O(g(n)), If there are positive constants n0 and c such that, to the right of n0 the f(n) always lies on or below c*g(n)
  - **f(n) <= c*g(n)** …… n ≥ n0
- O(g(n)) = { f(n) : There exist positive constant c and n0 such that 0 ≤ f(n) ≤ c g(n), for all n ≥ n0, c >0 }



After some **n0** value of **c*g(n)** is always greater than **f(n)**

f(n) = O(g(n)) … n>=n0, c>0, n0>1

Little-o Notation (o)

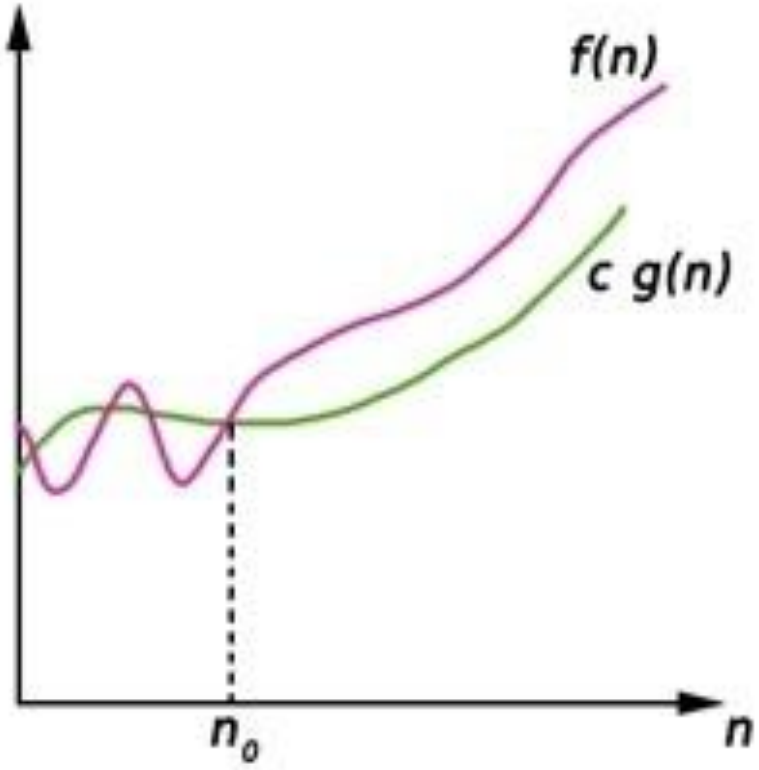
- **Little-o notation is used to denote an upper-bound that is not asymptotically tight**. It is formally defined as:
- o(g(n)) = { f(n) : There exist positive constant c and n0 such that $0 \leq f(n) \leq c\ g(n)$, for all $n \geq n0$, $c > 0$ }
- for any positive constant , there exists positive constant such that for all .
- **The set of functions f(n) are strictly smaller than c*g(n), meaning that little-o notation is a stronger upper bound than big-O notation**.
- In other words, the little-o notation does not allow the function to have the same growth rate as .

2. Omega Notation (Big Omega) Ω

- It defines the **best case** of an algorithm's time complexity, the Omega notation **defines whether the set of functions will grow faster or at the same rate as the expression**.
- It explains the **minimum amount of time** an algorithm requires to consider all input values.

Big-Omega Ω notation gives a **lower bound** for a function f(n) to within a constant factor.

- We write f(n) = Ω(g(n)), If there are positive constants n0 and c such that, to the right of n0 the f(n) always lies on or above c*g(n).
- Ω(g(n)) = { f(n) : There exist positive constant c and n0 such that 0 ≤ c g(n) ≤ f(n), for all n ≥ n0}
- **f(n) >= c g(n)** … n>=n0, c>0, n0>1



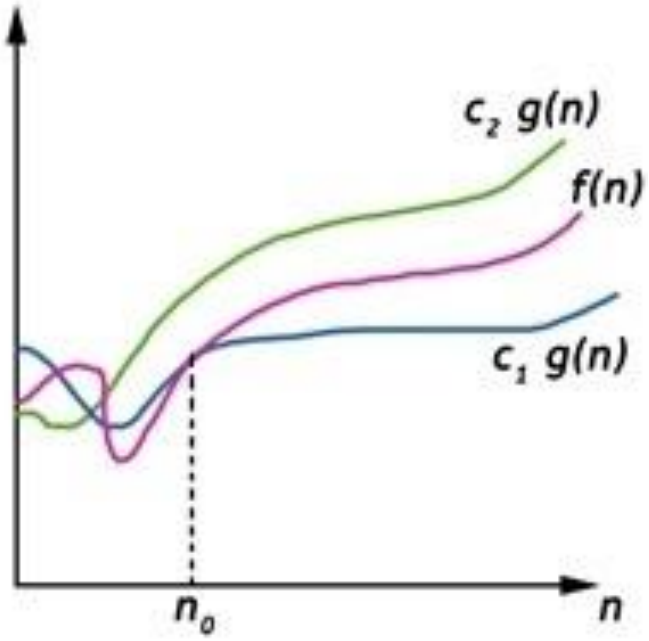
3. Theta Notation Θ

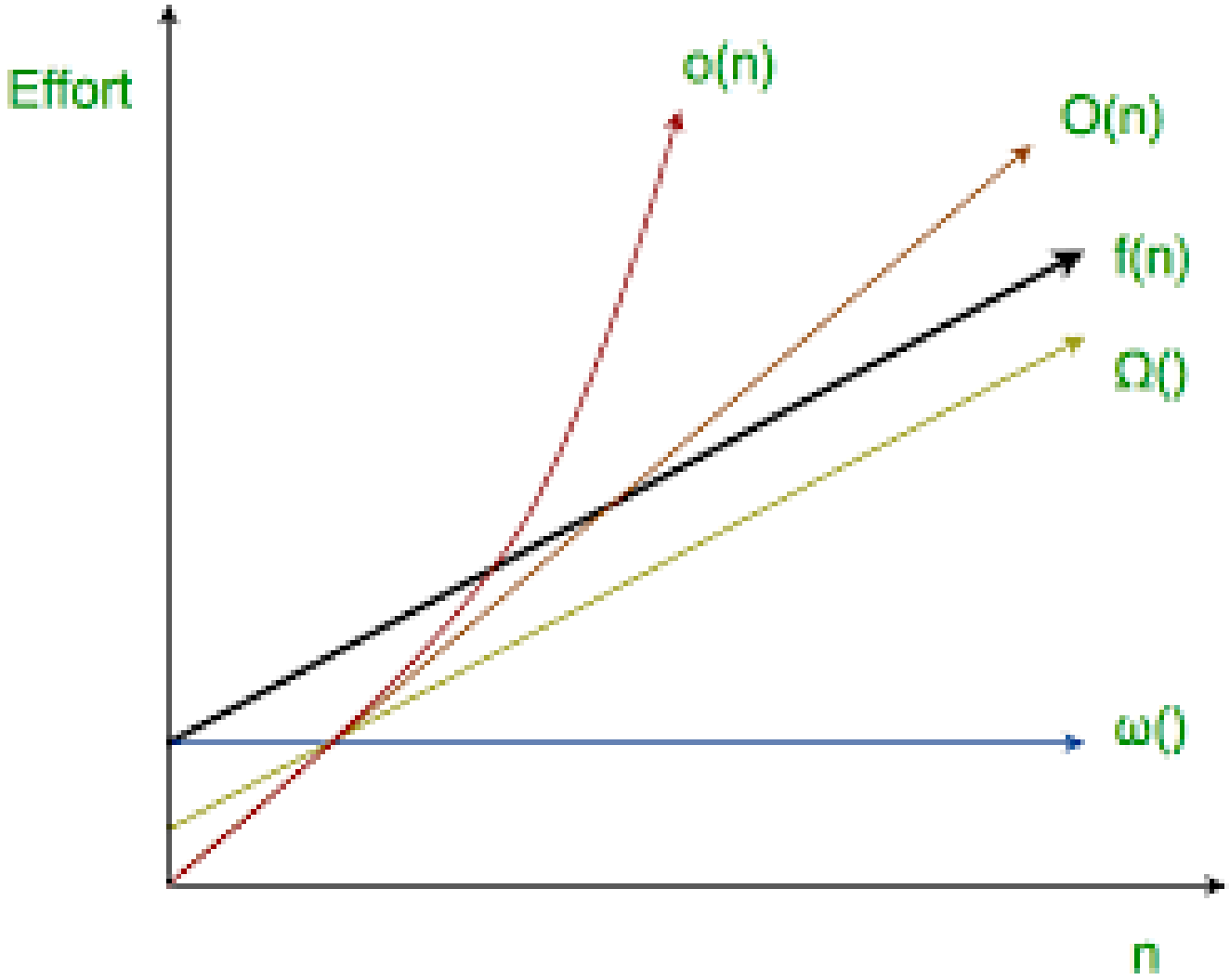
- It defines the **average case** of an algorithm's time complexity.
- Theta notation defines when the set of functions lies in both **O(expression)** and **Omega(expression)**, then Theta notation is used.

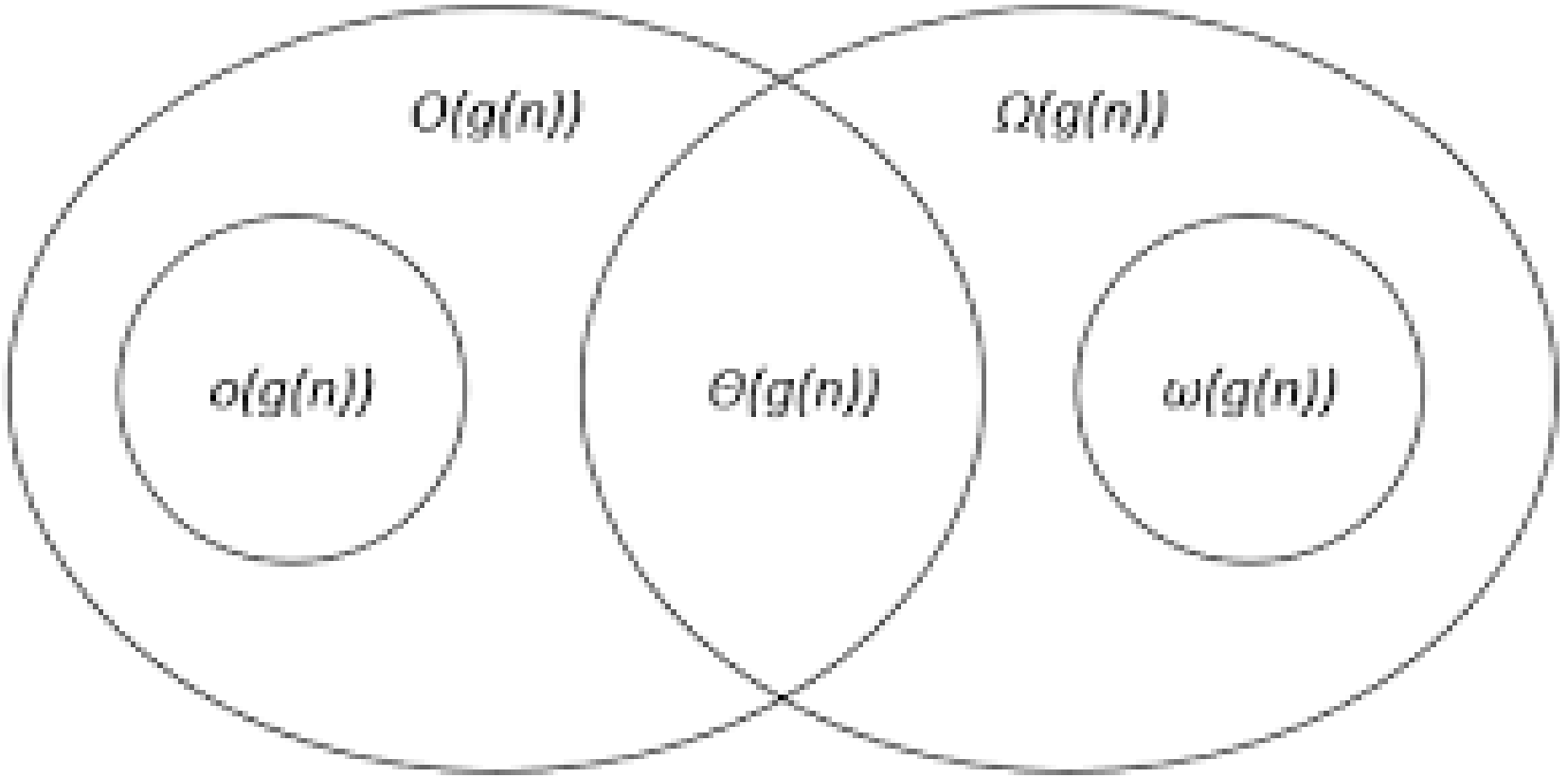
Big-Theta(Θ) notation gives bound for a function f(n) to within a constant factor.

- We write f(n) = Θ(g(n)), If there are positive constantsn0 and c1 and c2 such that, to the right of n0 the f(n) always lies between c1*g(n) and c2*g(n) inclusive.
- Θ(g(n)) = {f(n) : There exist positive constant c1, c2 and n0 such that 0 ≤ c1 g(n) ≤ f(n) ≤ c2 g(n), for all n ≥ n0}
- **c1 g(n) ≤ f(n) ≤ c2 g(n)**

...n>=n0, c1, c2 >0, n0>=1




Effort
o(n)
O(n)
f(n)
Ω()
ω()
n

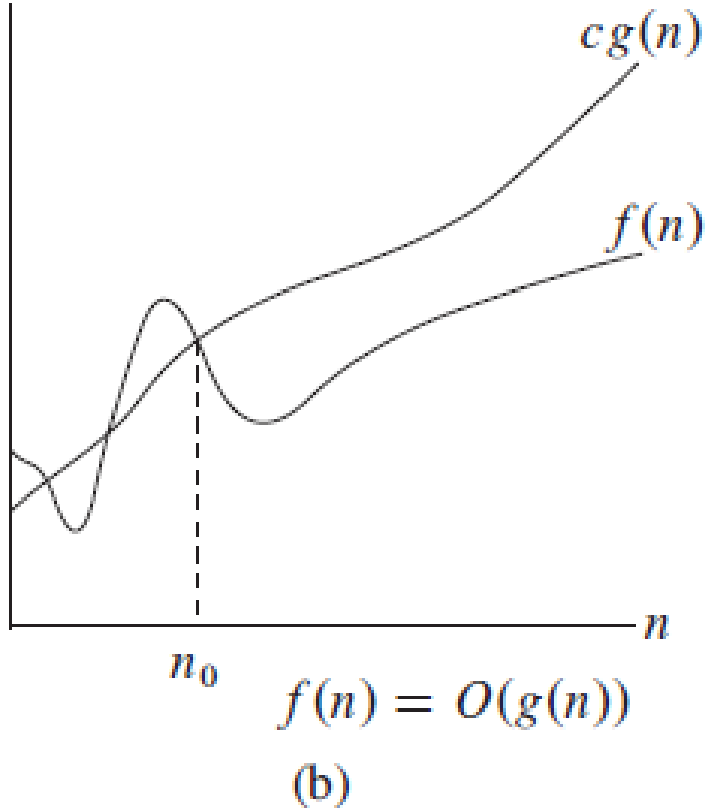

O(g(n))
Ω(g(n))
o(g(n))
Θ(g(n))
ω(g(n))

- Big-Oh Notation

$$o(g(n)) = \{f(n) : \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \le f(n) < cg(n) \text{ for all } n \ge n_0\}.$$

- Small oh -

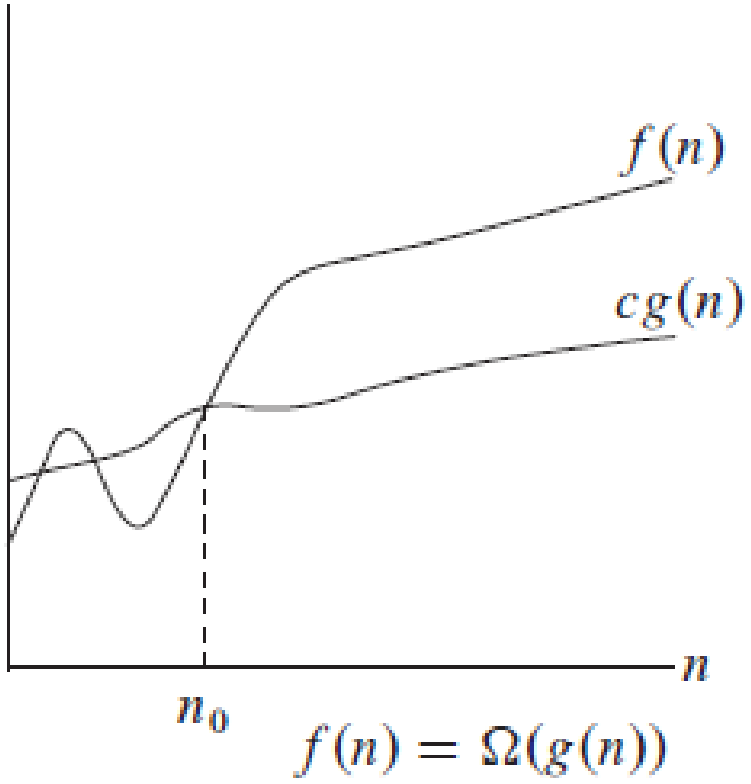
$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\}.$$



- **Omega Notation**

$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0\} .$$

$$\omega(g(n)) = \{f(n) : \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \le cg(n) < f(n) \text{ for all } n \ge n_0\} .$$

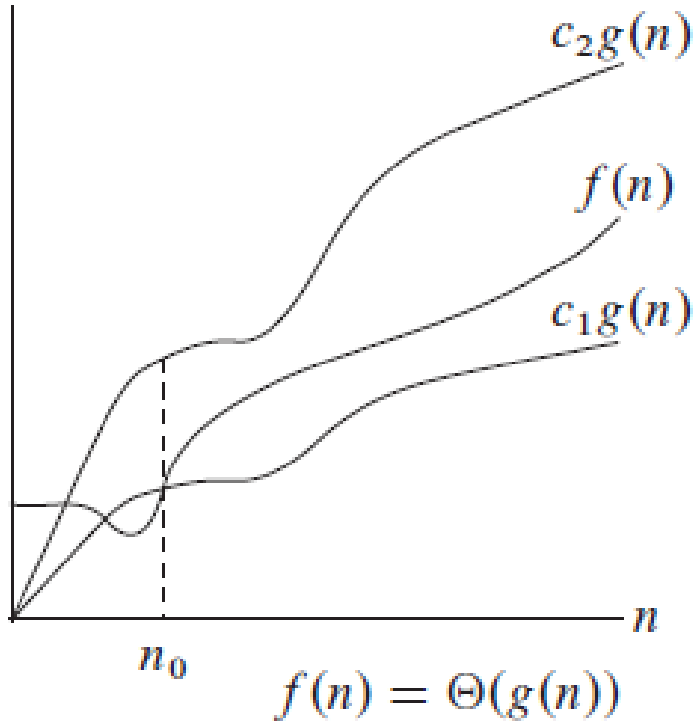


$f(n) = \Omega(g(n))$

(c)

- Theta Notation

$\Theta(g(n)) = \{f(n)$: there exist positive constants c_1, c_2, and n_0 such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0\}$.[1]



$f(n) = \Theta(g(n))$

(a)

Analysis of Control statements and loops

Consecutive Statements- constant time

```
int x  = 2;
int i;
x = x + 1;
```

Total time= 3 units

Nested Loops

```
for  ( i = 1; i < = n; i ++)
      for  ( j = 1; j < = n; j ++)
            //statement
```

Outer loop executes n times.

Inner loop executes n times.

Total time = n *n = O(n^2)

Loops

```
for  ( i = 1; i < = n; i ++)
     //statement
```

Loop executes n times.

Total time = O(n)

Best

Worst

O(1)	Constant
O(log n)	Logarithmic
O(n)	Linear
O(n^2)	Quadratic
O(n^3)	Cubic
O(n^k)	Polynomial
O(2^n)	Exponential
O(n!)	Factorial

Not in Syllabus 2024-25

Selection Sort

- The selection sort enhances the bubble sort by making **only a single swap for each pass** through the rundown.
- In order to do this, a selection sort searches for the biggest value as it makes a pass and, after finishing the pass, places it in the best possible area.
- Similarly, as with a bubble sort, after the first pass, the biggest item is in the right place. After the second pass, the following biggest is set up.
- This procedure proceeds and requires n-1 goes to sort n item since the last item must be set up after the (n-1)th pass.

Not in Syllabus 2024-25

How Selection Sort works

1.In the selection sort, first of all, we set the initial element as a **minimum**.

2.Now we will compare the minimum with the second element. If the second element turns out to be smaller than the minimum, we will swap them, followed by assigning to a minimum to the third element.

3.Else if the second element is greater than the minimum, which is our first element, then we will do nothing and move on to the third element and then compare it with the minimum. We will repeat this process until we reach the last element.

4.After the completion of each iteration, we will notice that our minimum has reached the start of the unsorted list.

5.For each iteration, we will start the indexing from the first element of the unsorted list. We will repeat the Steps from 1 to 4 until the list gets sorted or all the elements get correctly positioned.

Insertion Sort

Not in Syllabus 2024-25

```
INSERTION-SORT(A)
1  for j = 2 to A.length
2      key = A[j]
3      // Insert A[j] into the sorted
4      i = j − 1
5      while i > 0 and A[i] > key
6          A[i + 1] = A[i]
7          i = i − 1
8      A[i + 1] = key
```

Not in Syllabus 2024-25

INSERTION-SORT(A)	*cost*	*times*
1 **for** $j = 2$ **to** $A.length$	c_1	n
2 $key = A[j]$	c_2	$n-1$
3 // Insert $A[j]$ into the sorted sequence $A[1..j-1]$.	0	$n-1$
4 $i = j-1$	c_4	$n-1$
5 **while** $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
6 $A[i+1] = A[i]$	c_6	$\sum_{j=2}^{n}(t_j - 1)$
7 $i = i-1$	c_7	$\sum_{j=2}^{n}(t_j - 1)$
8 $A[i+1] = key$	c_8	$n-1$

For best case
Condition false, each time
Hence $t_j = 1$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1) .$$

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

Not in Syllabus 2024-25

INSERTION-SORT(A)	*cost*	*times*
1 **for** $j = 2$ **to** $A.length$	c_1	n
2 $key = A[j]$	c_2	$n-1$
3 // Insert $A[j]$ into the sorted sequence $A[1..j-1]$.	0	$n-1$
4 $i = j-1$	c_4	$n-1$
5 **while** $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
6 $A[i+1] = A[i]$	c_6	$\sum_{j=2}^{n}(t_j-1)$
7 $i = i-1$	c_7	$\sum_{j=2}^{n}(t_j-1)$
8 $A[i+1] = key$	c_8	$n-1$

For worst case
Condition true for max possible value, hence generate series

$$
\begin{aligned}
T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\
&\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\
&= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n \\
&\quad - (c_2 + c_4 + c_5 + c_8) .
\end{aligned}
$$

(n(n+1)/2)-1

Recursion

- **A function that calls itself.**
- Every recursive program **should have a termination condition**, otherwise program may go into the infinite loop.
- Recursion solves bigger problem in terms of smaller problems.
- For recursive function, number of parameters never change, code will never change, **only parameter values will changes**.
- Example: Fact(5) = 120
  = 5* Fact (4) ……. Reducing problem size
  = 5* 5* Fact (3)

Advantages of Recursion

- Recursion can reduce time complexity
- Adds clarity to code
- Reduce time to debug

Disadvantages of Recursion

- Uses more memory (stack in fact(5) example)
- Recursion can be slow

Recurrences

A recurrence is just a way of describing a problem using smaller versions of the same problem. Instead of writing a direct formula, we say:

“To solve a big input, first solve smaller inputs — then combine their results.”

So the time (or work) needed for size n depends on the time needed for smaller sizes.

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + cn & n > 1 \end{cases}$$

- Recurrence of Merge Sort: $T(n) = 2T(n/2) + n$
- Recurrence of Binary Search: $T(n) = T(n/2) + 1$

```
BINARY-SEARCH (A, low, high, x)
    if (low > high)
        return FALSE
    mid ← ⌊(low + high)/2⌋
    if x = A[mid]
        return TRUE
    if ( x < A[mid] )
        BINARY-SEARCH (A, low, mid-1, x)
    if ( x > A[mid] )
        BINARY-SEARCH (A, mid+1, high, x)
```

Recurrence equation example:

```
Void fun (int n)                    T(n)….. Sum of all steps inside equation
{
    if (n > 0 )                      1    ……  constant time
     {
         print (n)                   1  ……  constant time
         fun (n-1)                   T(n-1)
     }
}
```

$T(n) = T(n-1) + 2 = \quad T(n-1) + \ C$

Indirect Recurrence equation example:

```
fun (---)
{
    ----
    newfun(---)
    ----
}

newfun (---)
{
    ----
    fun(---)
    ----
}
```

Solving Recurrences

- Substitution method
- Master method / Master's Theorem
- Recursion Tree Method
- Iteration method…….not in syllabus

Substitution method

$$T(n) = nT(n-1)$$

Substitute $T(n-1)$:

$$T(n-1) = (n-1)T(n-2)$$

$$T(n) = n(n-1)T(n-2)$$

$$T(n-2) = (n-2)T(n-3)$$

$$T(n) = n(n-1)(n-2)T(n-3)$$

$$T(n) = n(n-1)(n-2)...3 \cdot 2 \cdot 1 \cdot T(0)$$

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 3 \times 2 \times 1$$

Time Complexity=O(n!)

Master's method

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$ ..... where, $a \geq 1$, $b > 1$, and $f(n) > 0$

- **n** is the **size** of the problem.
- **a** is the number of **subproblems** in the recursion. (Subproblems can not be in fraction)
- **n/b** is the **size of each subproblem**. (Here it is assumed that all subproblems are essentially the same size.)
- **f (n)** is the **sum of the work done** outside the recursive calls, which includes the **sum of dividing the problem and the sum of combining the solutions to the subproblems**. (f (n) can not be negative.)
- It is not possible always bound the function according to the requirement, so we make **three cases** which will tell us what kind of bound we can apply on the function.

Master Theorem

Recurrence Form

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^k \log^p n)$$

Where:

- $a \geq 1$
- $b > 1$
- $k \geq 0$
- p = Real number

Case 1: If $a > b^k$

$$T(n) = \Theta(n^{\log_b a})$$

Case 2: If $a = b^k$

(a) If $p > -1$

$$T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$$

(b) If $p = -1$

$$T(n) = \Theta(n^{\log_b a} \log \log n)$$

(c) If $p < -1$

$$T(n) = \Theta(n^{\log_b a})$$

Case 3: If $a < b^k$

(a) If $p \geq 0$

$$T(n) = \Theta(n^k \log^p n)$$

(b) If $p < 0$

$$T(n) = \Theta(n^k)$$

Master's method

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$ ..... where, $a \geq 1$, $b > 1$, and $f(n) > 0$

- **n** is the **size** of the problem.
- **a** is the number of **subproblems** in the recursion. (Subproblems can not be in fraction)
- **n/b** is the **size of each subproblem**. (Here it is assumed that all subproblems are essentially the same size.)
- **f (n)** is the **sum of the work done** outside the recursive calls, which includes the **sum of dividing the problem and the sum of combining the solutions to the subproblems**. (f (n) can not be negative.)
- It is not possible always bound the function according to the requirement, so we make **three cases** which will tell us what kind of bound we can apply on the function.

Examples

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^k \log^p n)$$

Where:

- $a \geq 1$
- $b > 1$
- $k \geq 0$
- p = Real number

Problem

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Step 1: Identify Parameters

$$a = 4, \quad b = 2$$

Given:

$$f(n) = n = n^1$$

So,

$$k = 1, \quad p = 0$$

Step 2: Compare a and b^k

$$b^k = 2^1 = 2$$

$$a = 4$$

Since:

$$4 > 2$$

$$a > b^k$$

Step 3: Apply Master Theorem (Case 1)

$$T(n) = \Theta(n^{\log_b a})$$

$$= \Theta(n^{\log_2 4})$$

$$= \Theta(n^2)$$

Master Theorem Example Case -II

1. $T(n) = aT\left(\frac{n}{b}\right) + f(n)$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

So:

- $a = 2$
- $b = 2$
- $f(n) = n \log n$

2. $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$

$$a = 2, \quad b = 2, \quad f(n) = \frac{n}{\log n}$$

$$f(n) = \frac{n}{\log n} = n \cdot \log^{-1} n$$

k=1

p=−1

$$T(n) = \Theta\left(n \cdot \log^{(-1+1)} n\right) = \Theta(n \cdot \log^0 n) = \Theta(n)$$

3. $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log^2 n}$

$$a = 2, \quad b = 2, \quad f(n) = \frac{n}{\log^2 n}$$

$$f(n) = \frac{n}{(\log n)^2} = n \log^{-2} n$$

So:

- $k = 1$
- $p = -2$

Master Theorem Example Case -III

1 $$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

$$a = 2, \quad b = 2, \quad f(n) = n^2$$

2 $$T(n) = 3T\left(\frac{n}{2}\right) + \frac{n^2}{\log n}$$

$$a = 3, \quad b = 2, \quad f(n) = \frac{n^2}{\log n}$$

- $f(n) = \frac{n^2}{\log n} = n^2 \log^{-1} n$

Recursion Tree Method

$$T(n) = 2T(\frac{n}{2}) + n$$

No. of children

Draw recurrence tree and calculate time taken by each level.

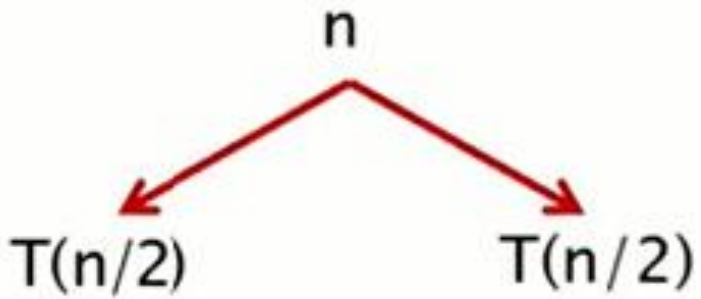
Sum the work done by all the levels.

f(n) : time require to break n into 2 parts, f(n) : always root node of tree

1. Recursion Tree Method is a pictorial representation of an iteration method which is in the form of a tree where at each level nodes are expanded.
2. It is useful when the divide & Conquer algorithm is used.
3. It is sometimes difficult to come up with a good guess. In Recursion tree, each root and child represents the cost of a single subproblem.
4. We sum the costs within each of the levels of the tree to obtain a set of pre-level costs and then sum all pre-level costs to determine the total cost of all levels of the recursion.
5. A Recursion Tree is best used to generate a good guess, which can be verified by the Substitution Method.

Examples

$$T(n) = 2T(\frac{n}{2}) + n$$



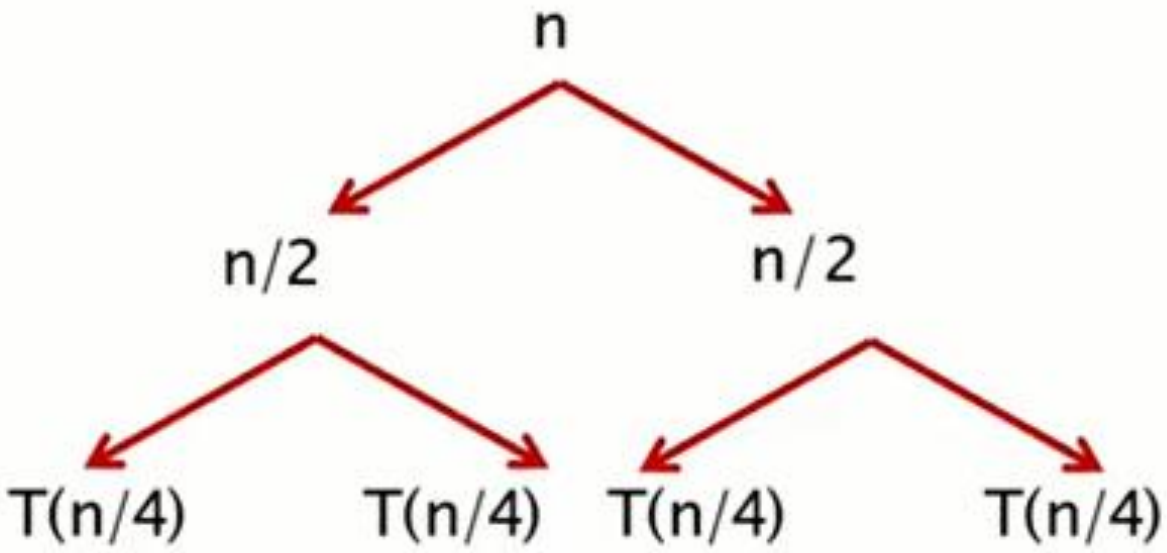
$$put\ n = \frac{n}{2}$$

$$T(\frac{n}{2}) = 2T(\frac{n}{2^2}) + \frac{n}{2}$$

$$T(n) = 2T(\frac{n}{2}) + n$$

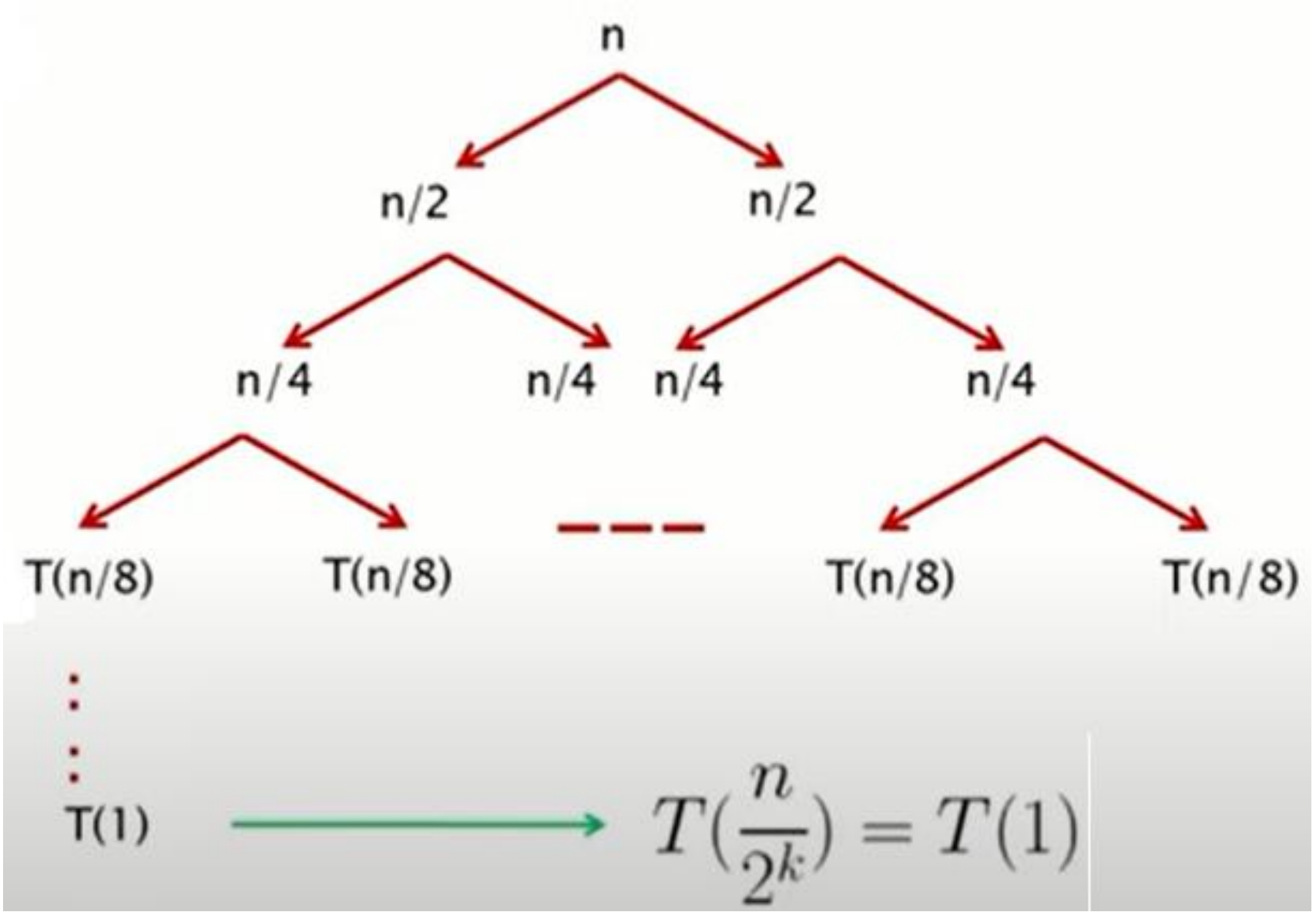
$$put\ n = \frac{n}{2}$$

$$T(\frac{n}{2}) = 2T(\frac{n}{2^2}) + \frac{n}{2}$$



Expand the tree until it reaches the base condition of the recursion T(1).

$$T(n) = 2T(\frac{n}{2}) + n$$



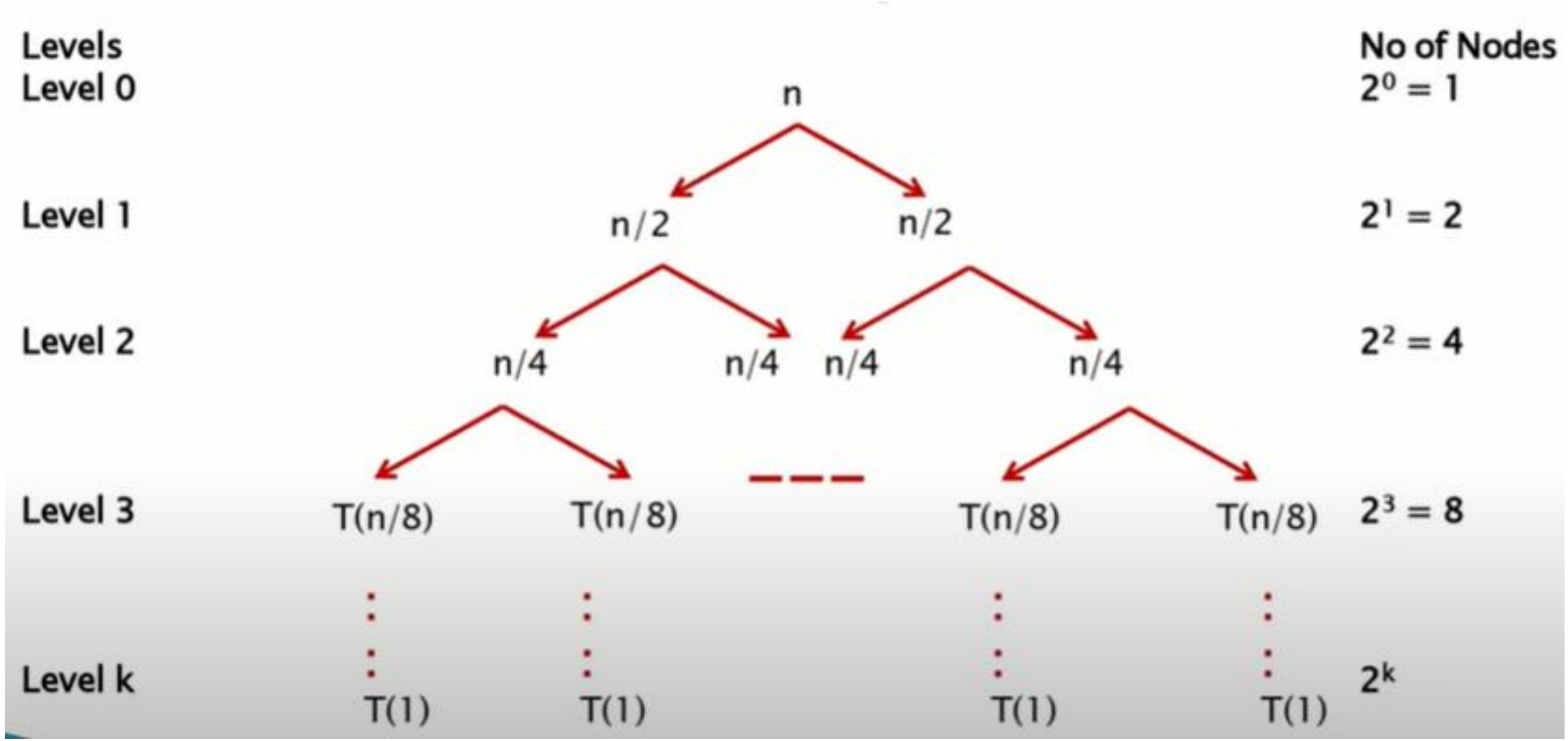
$$T(\frac{n}{2^k}) = T(1)$$

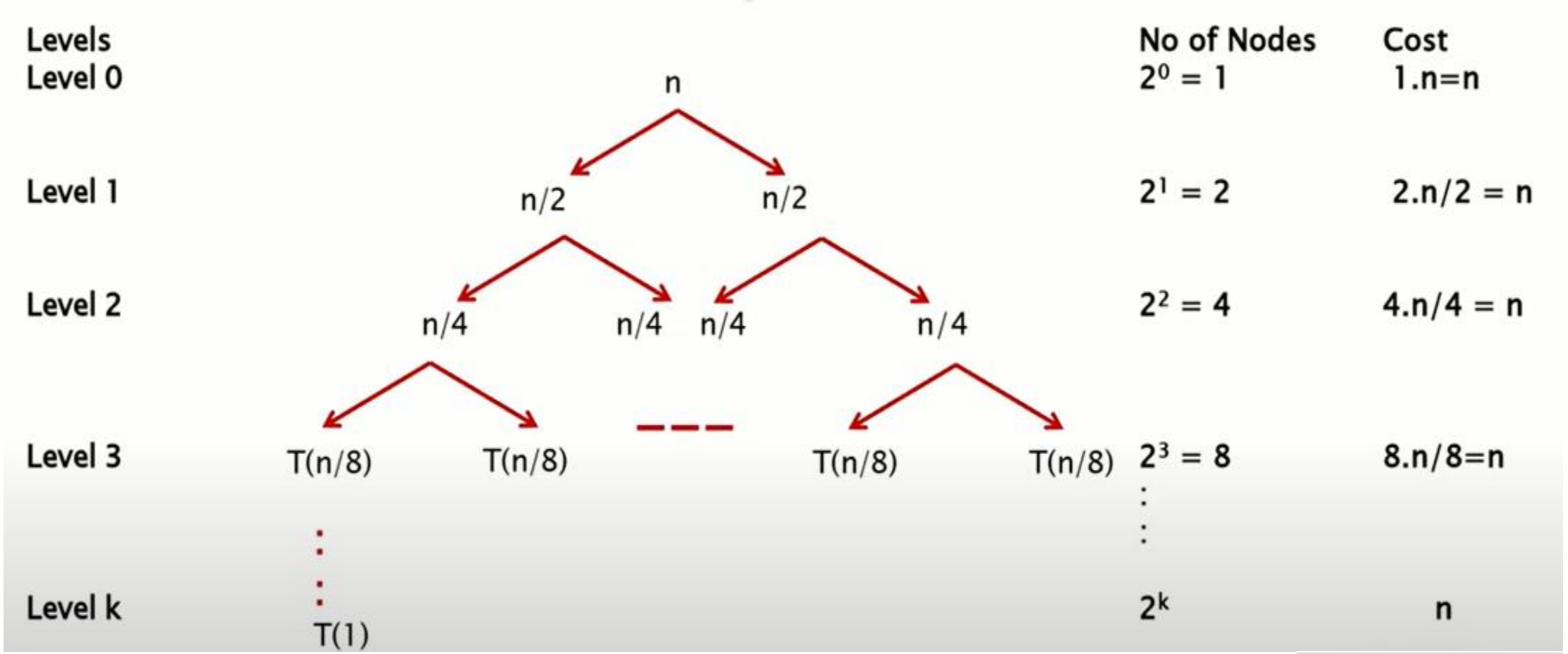
$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$Taking\ log_2\ both\ the\ sides$$

$$k = log_2 n$$


Levels
No of Nodes
Level 0
n
$2^0 = 1$
Level 1
n/2
n/2
$2^1 = 2$
Level 2
n/4
n/4
n/4
n/4
$2^2 = 4$
Level 3
T(n/8)
T(n/8)
T(n/8)
T(n/8)
$2^3 = 8$
Level k
T(1)
T(1)
T(1)
T(1)
2^k



Substituting the value
$T(n) = \theta(n.\log n)$

The recursion-tree method

Convert the recurrence into a tree:

- Each node represents the cost incurred at various levels of recursion
- Sum up the costs of all levels

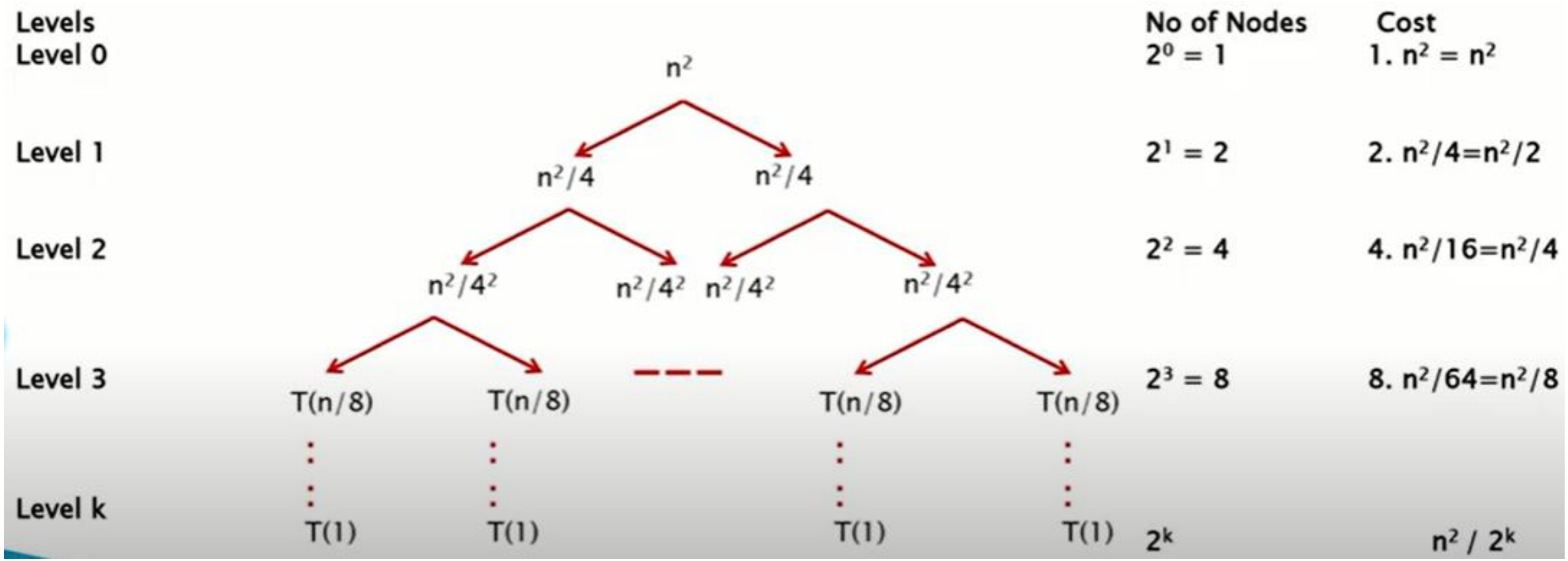
Used to “guess” a solution for the recurrence

Example :

$$T(n) = 2T(\frac{n}{2}) + n^2$$

$$put\ n = \frac{n}{2}$$

$$T(\frac{n}{2}) = 2T(\frac{n}{2^2}) + \left(\frac{n}{2}\right)^2$$


Levels
No of Nodes
Cost
Level 0
$2^0 = 1$
1. $n^2 = n^2$
n^2
Level 1
$2^1 = 2$
2. $n^2/4 = n^2/2$
$n^2/4$
$n^2/4$
Level 2
$2^2 = 4$
4. $n^2/16 = n^2/4$
$n^2/4^2$
$n^2/4^2$
$n^2/4^2$
$n^2/4^2$
Level 3
$2^3 = 8$
8. $n^2/64 = n^2/8$
T(n/8)
T(n/8)
T(n/8)
T(n/8)
Level k
T(1)
T(1)
T(1)
T(1)
2^k
$n^2 / 2^k$

$$= n^2 . \left[\left(\frac{1}{2}\right)^0 + \left(\frac{1}{2}\right)^1 + \left(\frac{1}{2}\right)^2 + - - - - + \left(\frac{1}{2}\right)^{k-1} \right]$$

$$= n^2 . \left[\frac{1}{1 - \frac{1}{2}} \right]$$

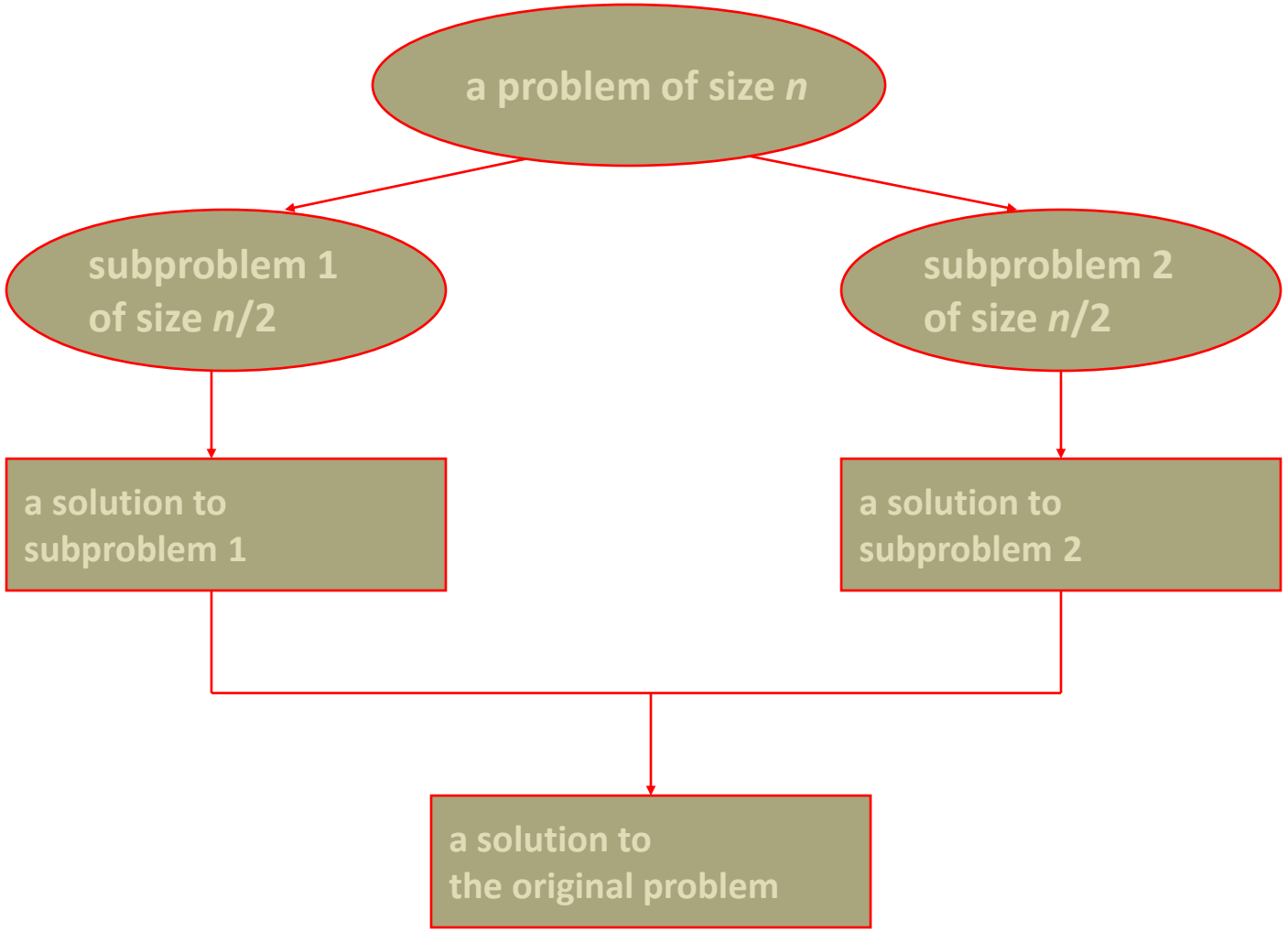
$$= 2n^2$$

$$\therefore T(n) = \Theta(n^2)$$

Divide-and-Conquer

- ***Divide and Conquer*** is a method of algorithm design that has created such efficient algorithms as Merge Sort.
- three distinct steps:
  - **Divide**: If the input size is too large to deal with in a straightforward manner, divide the data into two or more disjoint subsets.
  - **Recur**: Use divide and conquer to solve the sub problems associated with the data subsets.
  - **Conquer**: Take the solutions to the sub problems and “merge” these solutions into a solution for the original problem.

Divide-and-conquer Technique



Divide & Conquer Algorithm

1. Algo D and C(P)
2. {
3. If small (P) then return S(P)
4. else
5. {
6. Divide P into smaller instances P_1,P_2 ….P_k , k>=1
7. Apply D and C to each of these problem
8. Return combine (D and C(P_1), D and C(P_2)…D and C(P_k);
9. }
10. }

Min-Max Algorithm

```
MaxMin (i, j, max, min)
{   if (i = j)  then   max = min = a[i]
    else if (i = j-1) then
            {    if (a[i] < a[j] ) then { max =a[j]; min=a[i]; }
                 else { max=a[i]; min=a[j]; }
            }
    else
    {     mid = [ ( i+j ) /2]
          MaxMin (i, mid, max, min);
          MaxMin (mid+1, j, max1, min1);
          if (max < max1) then max =max1;
          if (min > min1) Then min=min1;
    }
}
```

Merge-Sort

- Algorithm:
  - **Divide**: If S has at leas two elements (nothing needs to be done if S has zero or one elements), remove all the elements from S and put them into two sequences, S_1 and S_2, each containing about half of the elements of S. (i.e. S_1 contains the first $\lceil n/2 \rceil$ elements and S_2 contains the remaining $\lfloor n/2 \rfloor$ elements.
  - **Recur**: Recursive sort sequences S_1 and S_2.
  - **Conquer**: Put back the elements into S by merging the sorted sequences S_1 and S_2 into a unique sorted sequence.

Merge-Sort Algorithm

- **Divide**: Divide the n-element sequence to be sorted into two subsequences of n=2 elements each.
- **Conquer**: Sort the two subsequences recursively using merge sort.
- **Combine**: Merge the two sorted subsequences to produce the sorted answer

Merge-Sort

- Merge Sort Tree:
  - Take a binary tree T
  - Each node of T represents a recursive call of the merge sort algorithm.
  - We associate with each node v of T a the set of input passed to the invocation v represents.
  - The external nodes are associated with individual elements of S, upon which no recursion is called.

Merge Sort

The following figure illustrates the dividing (splitting) procedure.

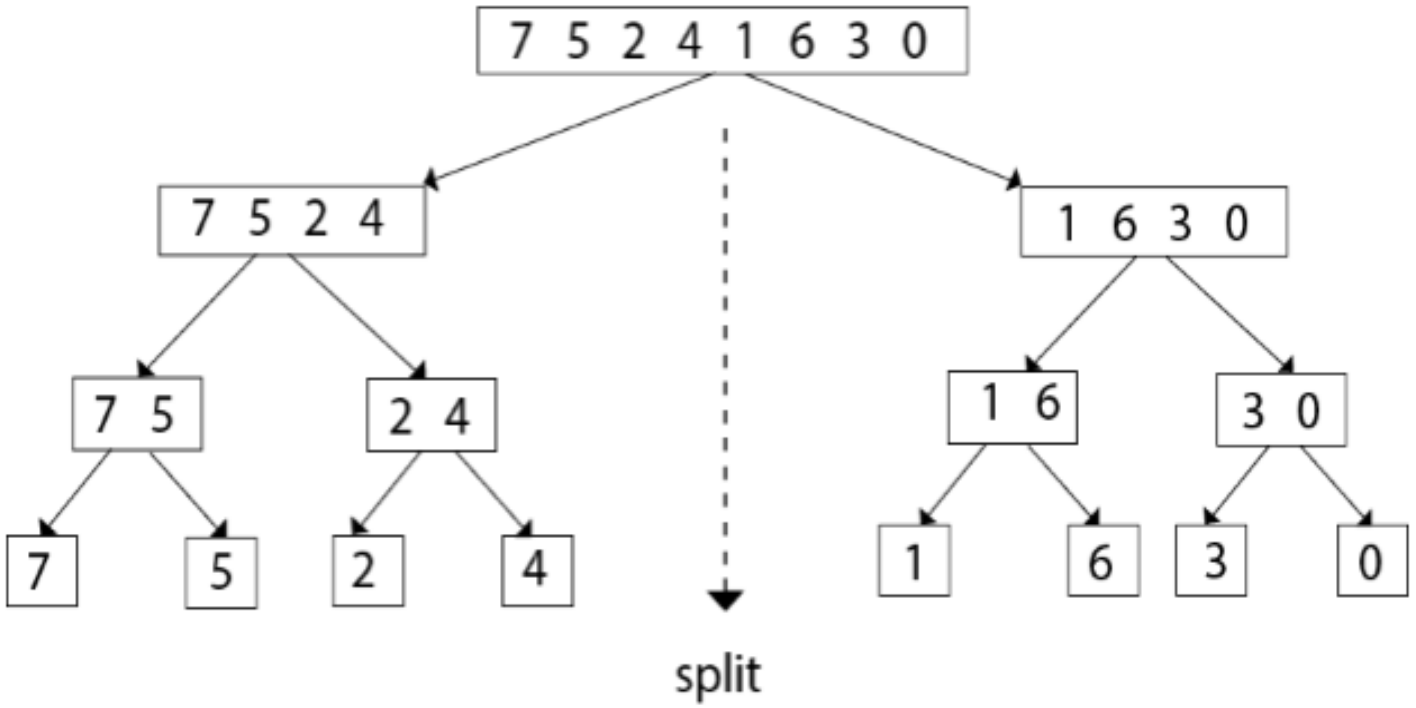


Figure: Merge sort divide phase

Merge Sort

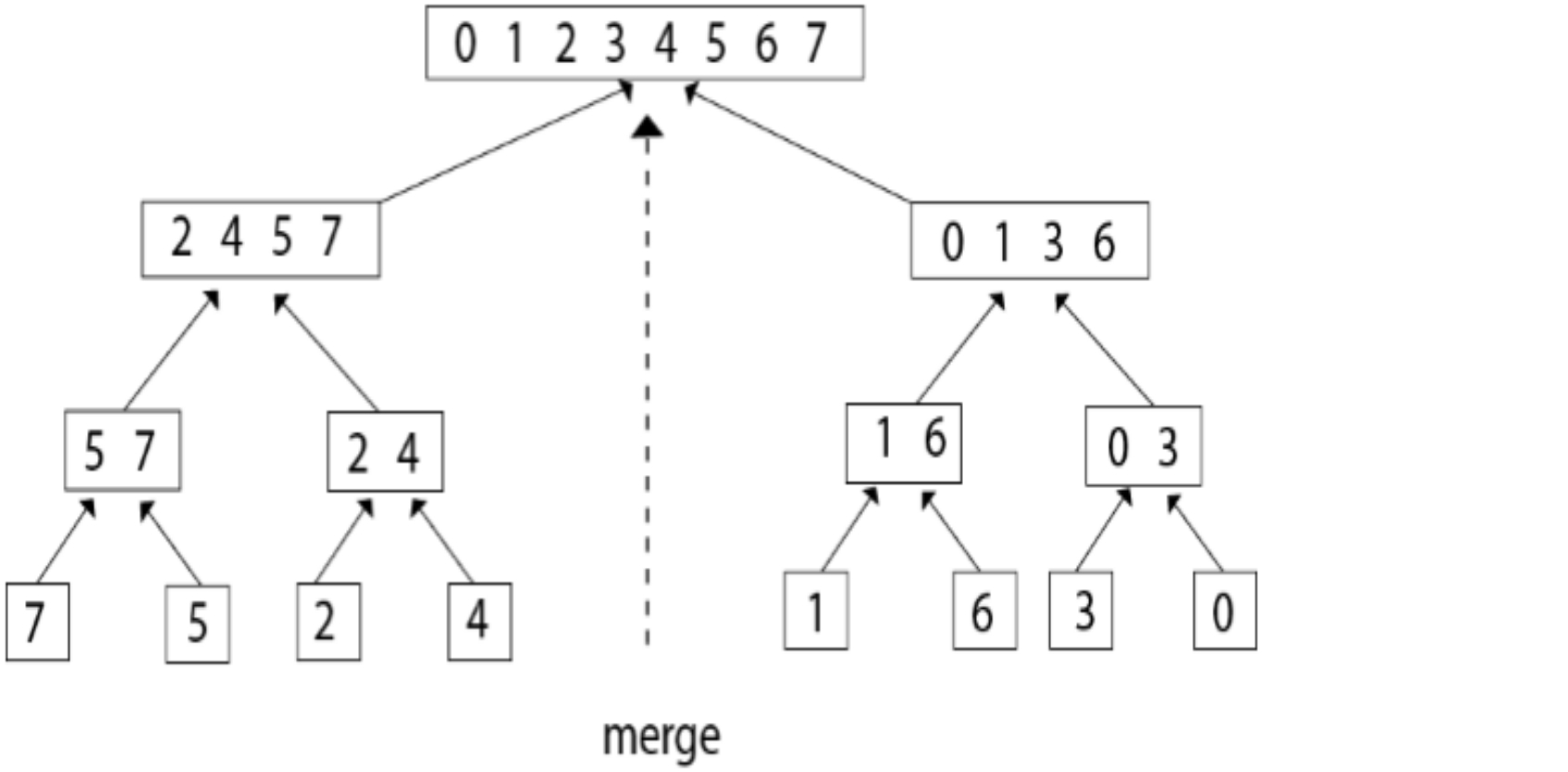
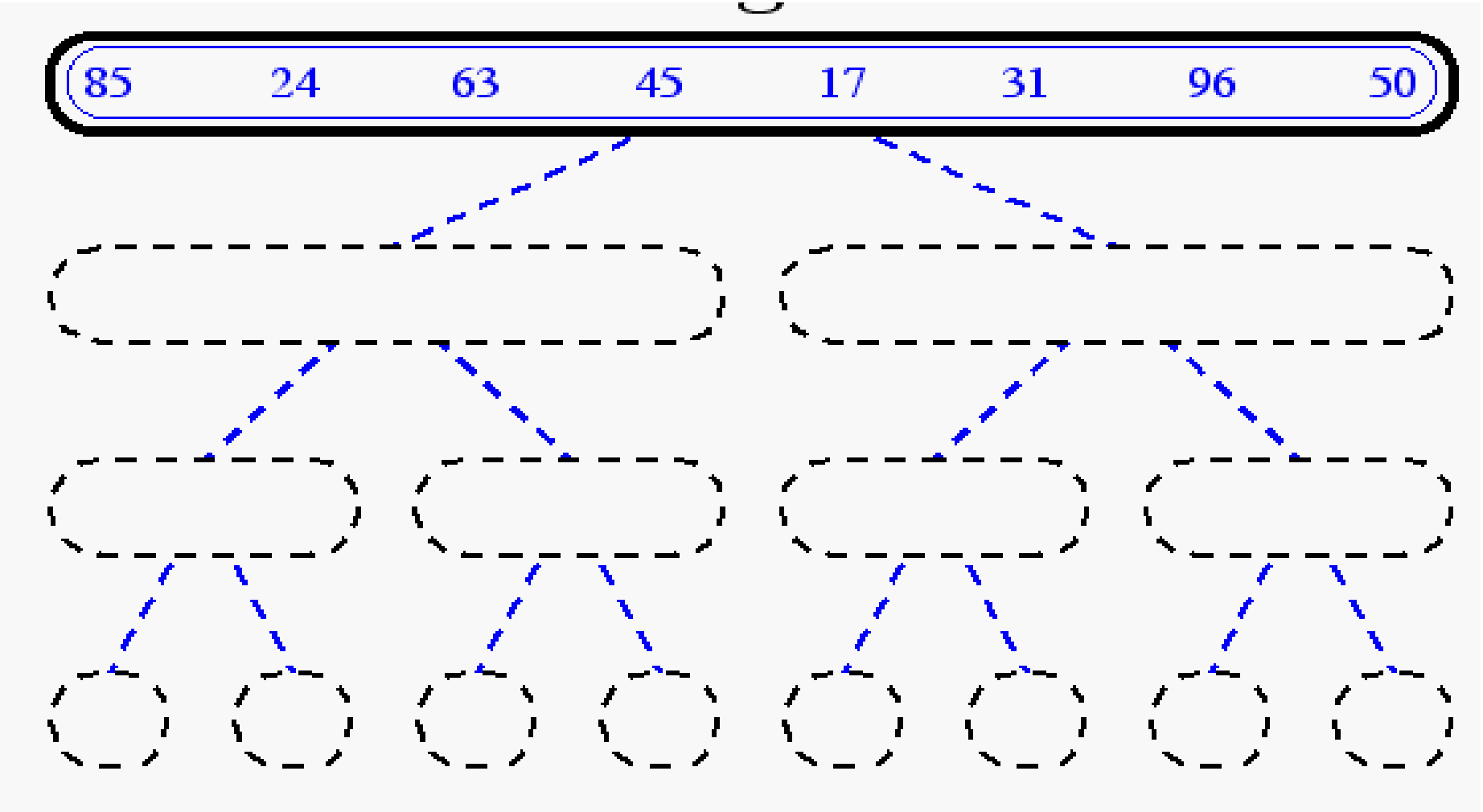
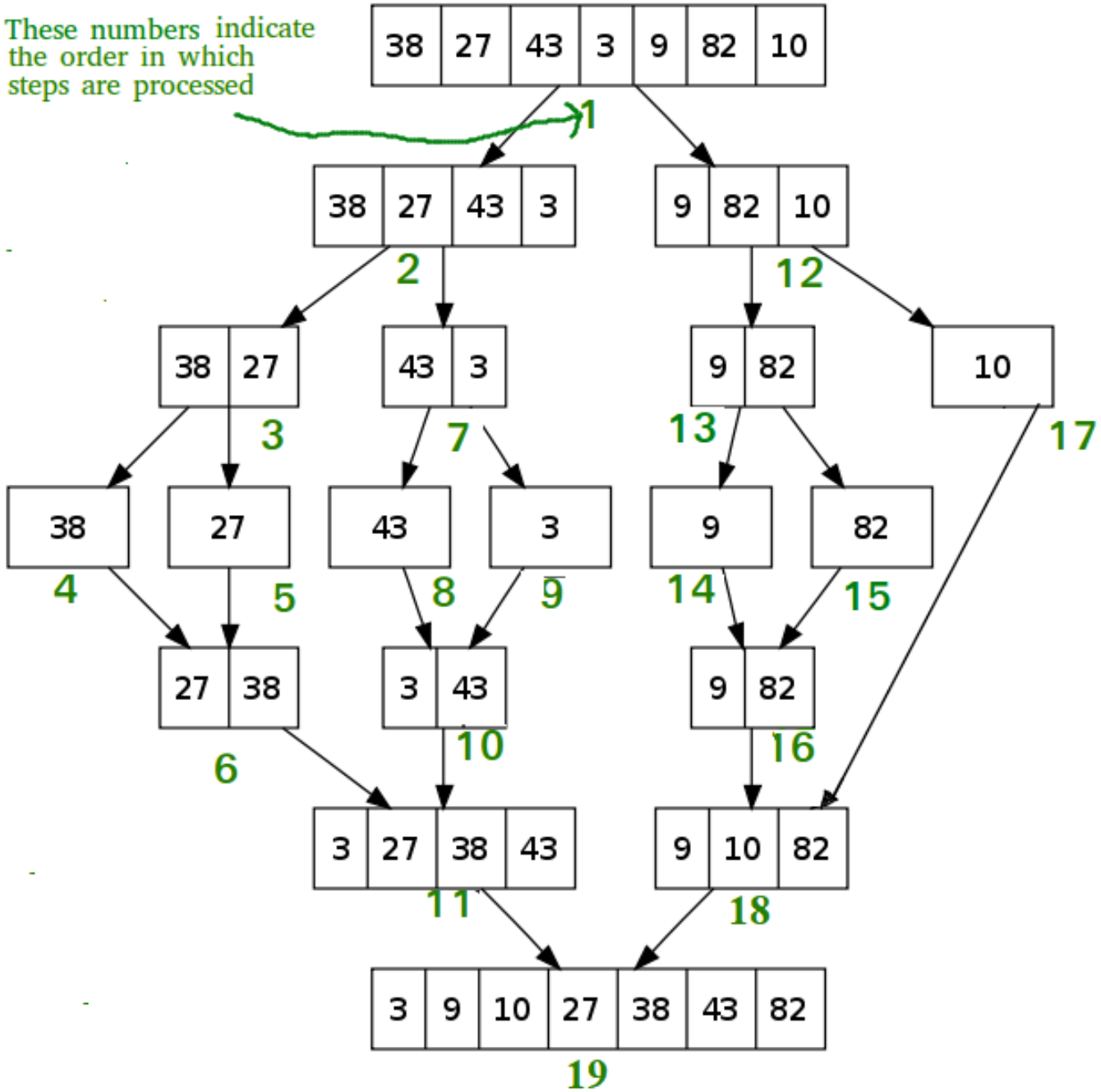


Fig: Merge sort: combine phase

Merge-Sort



Merge-Sort


These numbers indicate the order in which steps are processed
38 27 43 3 9 82 10
1
38 27 43 3
9 82 10
2
12
38 27
43 3
9 82
10
3
7
13
17
38
27
43
3
9
82
4
5
8
9
14
15
27 38
3 43
9 82
10
16
6
3 27 38 43
9 10 82
11
18
3 9 10 27 38 43 82
19

Quick-Sort

- Another divide-and-conquer sorting algorithm
- To understand quick-sort, let's look at a high-level description of the algorithm

1) **Divide** : If the sequence S has 2 or more elements, select an element x from S to be your **pivot**. Any arbitrary element, like the last, will do. Remove all the elements of S and divide them into 3 sequences:

   L, holds S's elements less than x

   E, holds S's elements equal to x

   G, holds S's elements greater than x

2) **Recurse**: Recursively sort L and G

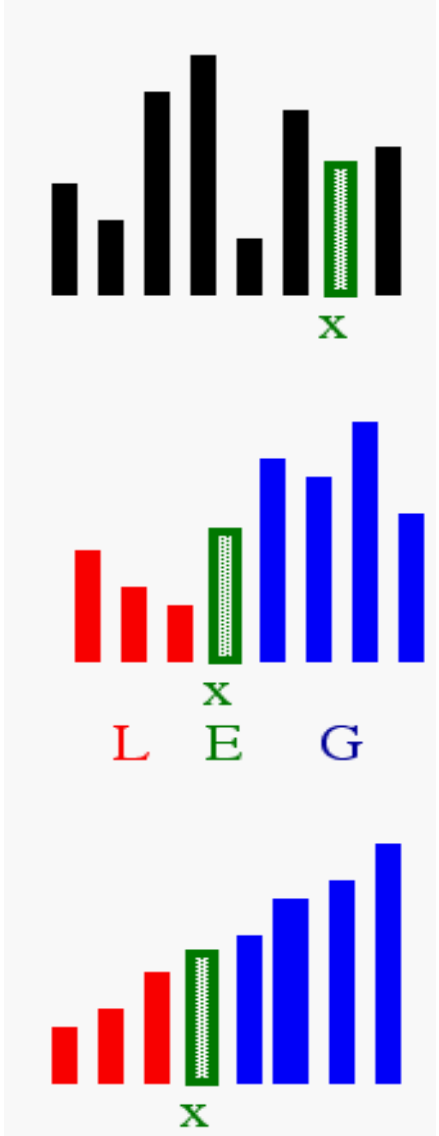
3) **Conquer**: Finally, to put elements back into S in order, first inserts the elements of L, then those of E, and those of G.

Idea of Quick Sort

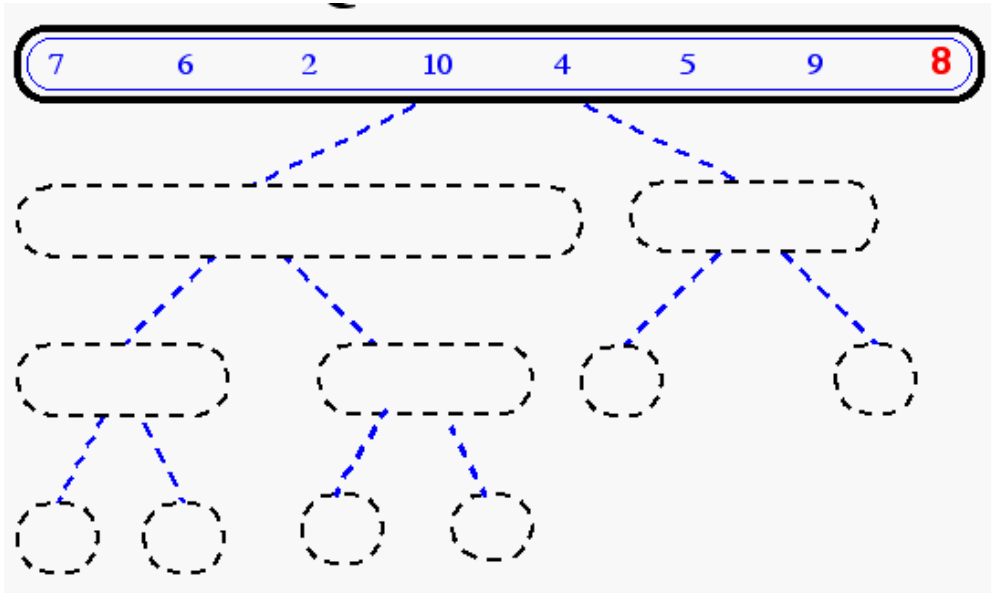
1) **Select**: pick an element

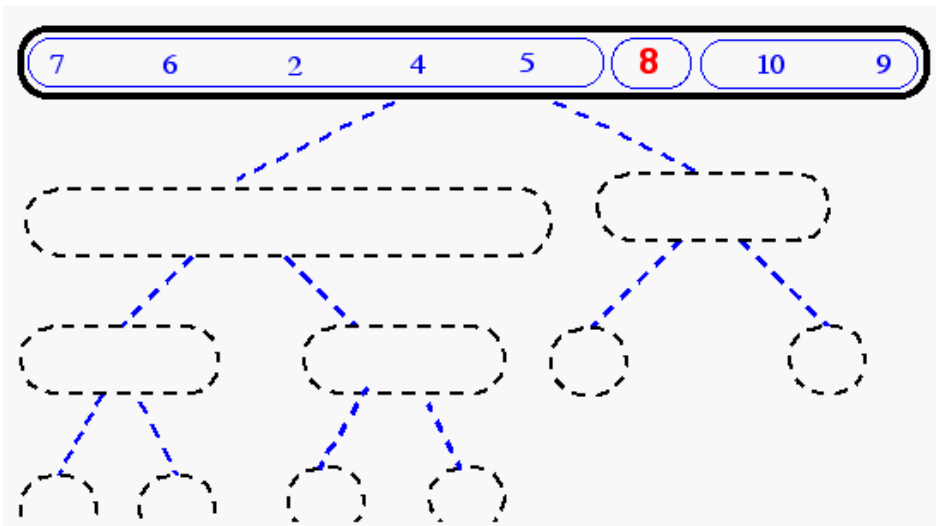
2**) Divide**: rearrange elements so that x goes to its final position E

3) **Recurse and Conquer**: recursively sort



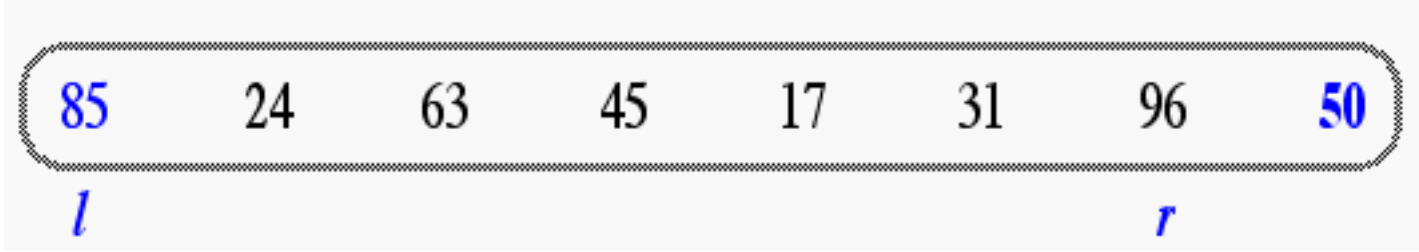
Quick-Sort Tree



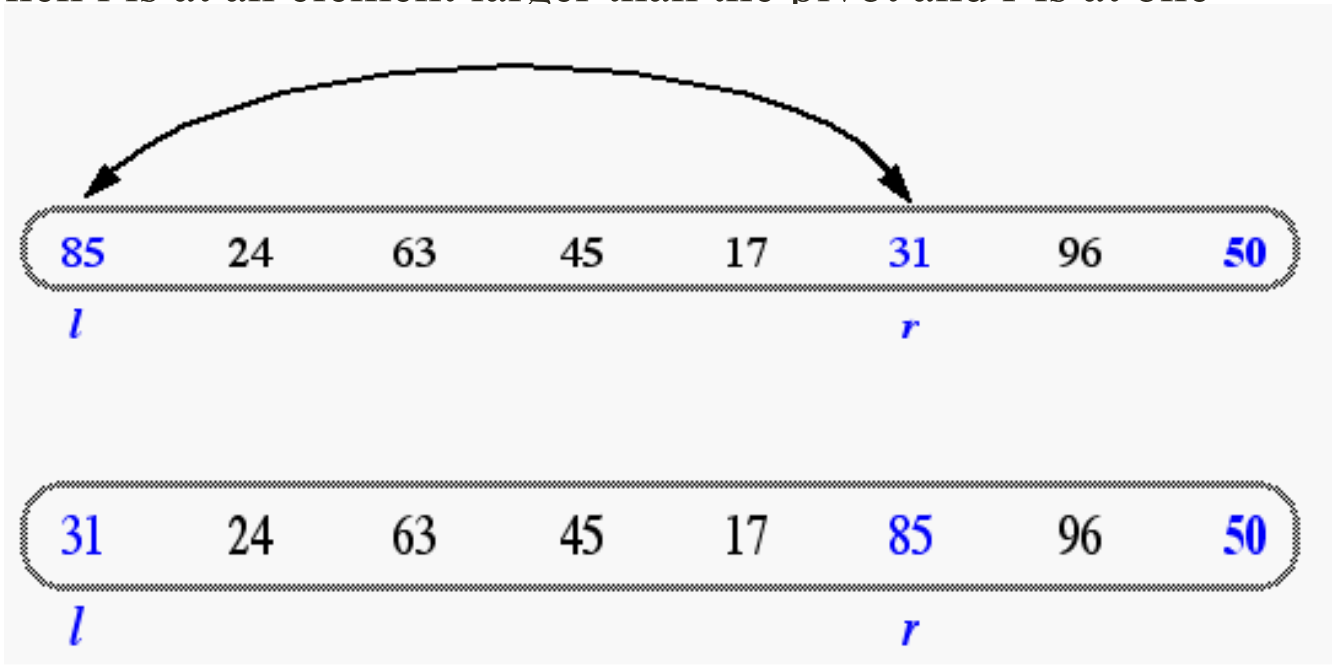


In-Place Quick-Sort

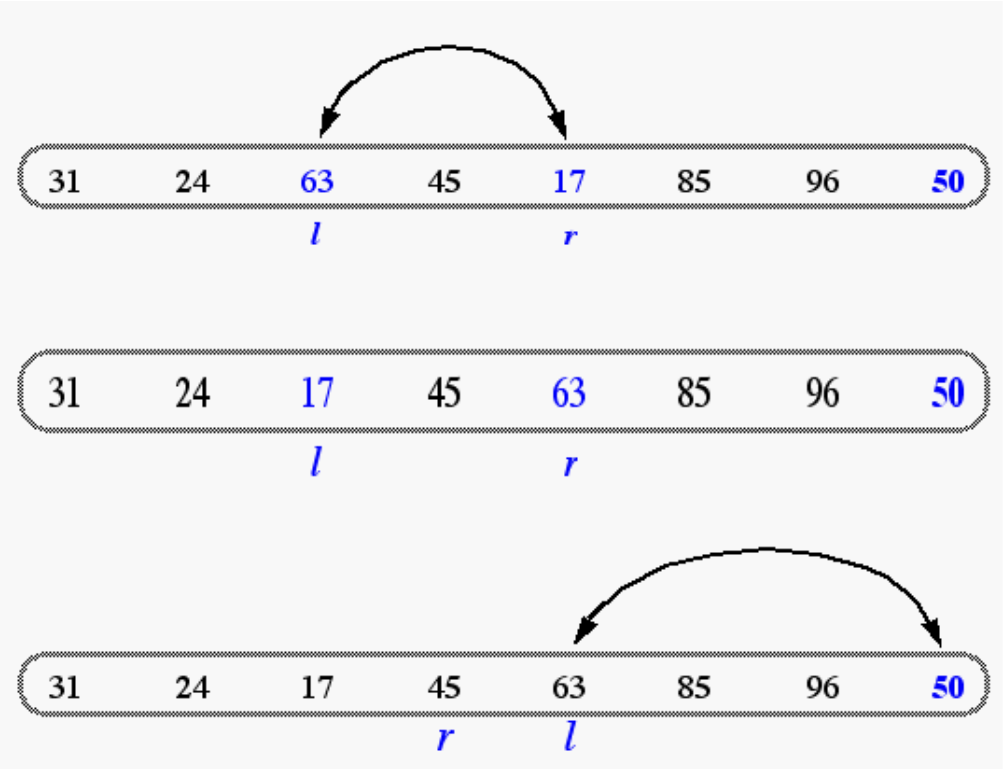
Divide step: l scans the sequence from the left, and r from the right.



A swap is performed when l is at an element larger than the pivot and r is at one smaller than the pivot.



In Place Quick Sort (cont'd)



A final swap with the pivot completes the divide step

Quick-Sort Algorithm

```
Partition(A, p, r)
x = A[r]        // pivot
i = p - 1

for j = p to r – 1
{
   if A[j] ≤ x
{
        i = i + 1
        exchange A[i] & A[j]
}
}
exchange A[i+1] & A[r]
return i + 1
```

```
QUICKSORT(A, p, r)
1   if p < r
2       then q ← PARTITION(A, p, r)
3            QUICKSORT(A, p, q − 1)
4            QUICKSORT(A, q + 1, r)
```

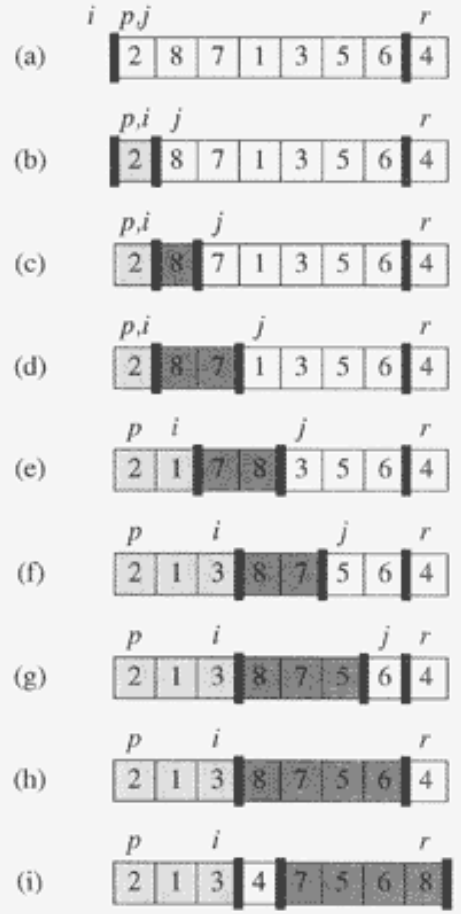


Figure 7.1 The operation of PARTITION on a sample array. Lightly shaded array elements are all in the first partition with values no greater than x. Heavily shaded elements are in the second partition with values greater than x. The unshaded elements have not yet been put in one of the first two partitions, and the final white element is the pivot. **(a)** The initial array and variable settings. None of the elements have been placed in either of the first two partitions. **(b)** The value 2 is "swapped with itself" and put in the partition of smaller values. **(c)–(d)** The values 8 and 7 are added to the partition of larger values. **(e)** The values 1 and 8 are swapped, and the smaller partition grows. **(f)** The values 3 and 8 are swapped, and the smaller partition grows. **(g)–(h)** The larger partition grows to include 5 and 6 and the loop terminates. **(i)** In lines 7–8, the pivot element is swapped so that it lies between the two partitions.

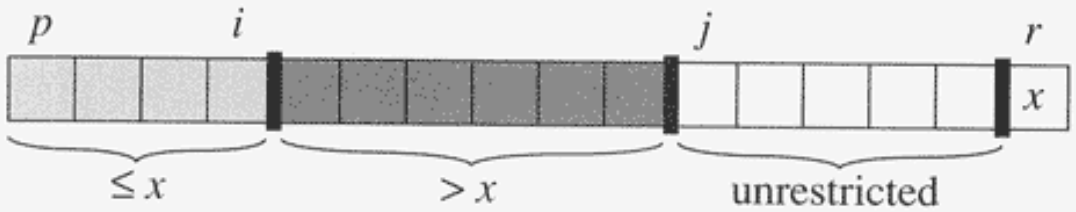


Figure 7.2 The four regions maintained by the procedure PARTITION on a subarray $A[p\,..\,r]$. The values in $A[p\,..\,i]$ are all less than or equal to x, the values in $A[i+1\,..\,j-1]$ are all greater than x, and $A[r] = x$. The values in $A[j\,..\,r-1]$ can take on any values.

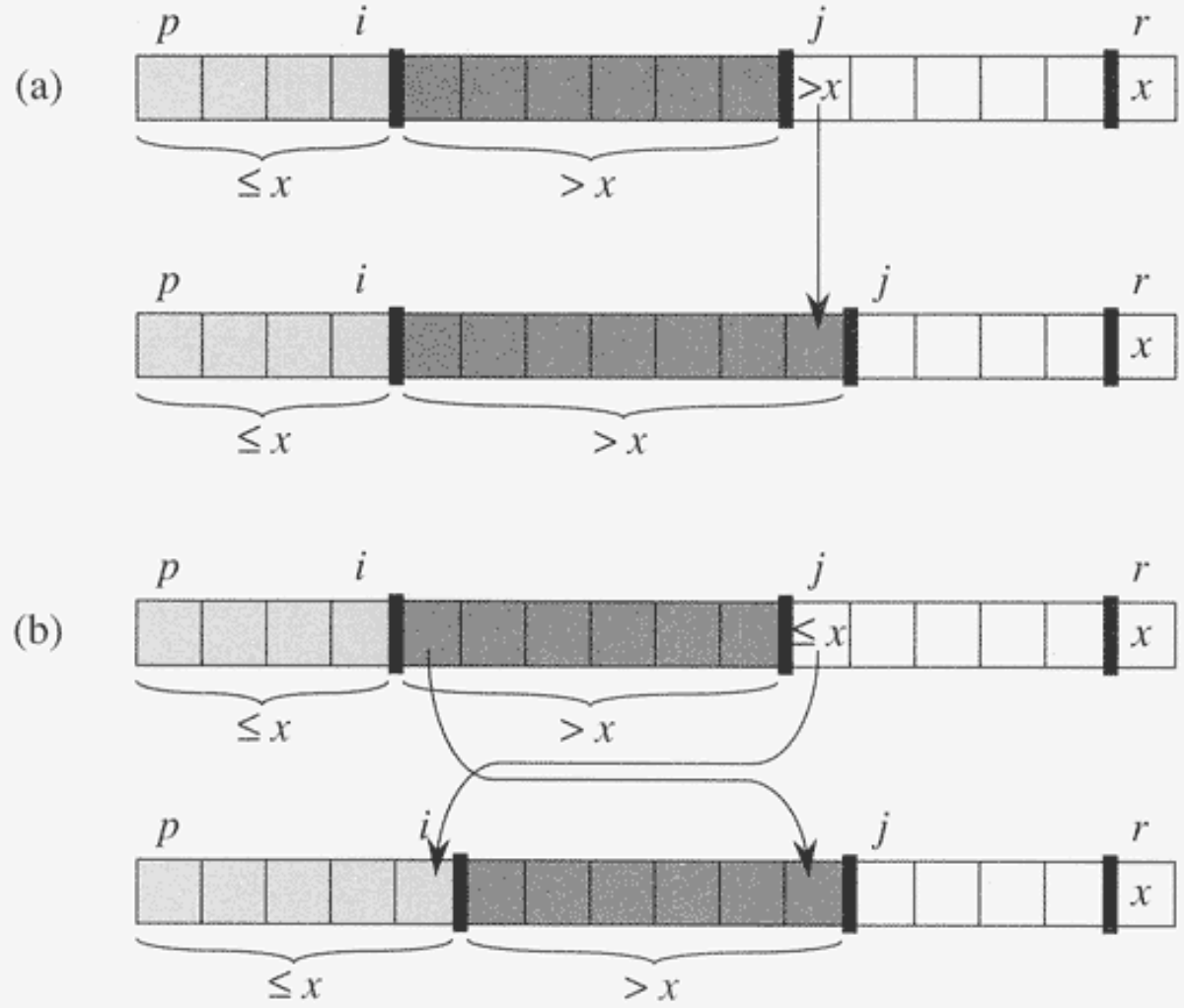


Figure 7.3 The two cases for one iteration of procedure PARTITION. **(a)** If $A[j] > x$, the only action is to increment j, which maintains the loop invariant. **(b)** If $A[j] \leq x$, index i is incremented, $A[i]$ and $A[j]$ are swapped, and then j is incremented. Again, the loop invariant is maintained.

Karatsuba Algorithm

- **Easy Way to Multiply Big Numbers**
- Karatsuba is a fast multiplication method.
- It is used to multiply big numbers.
- It is faster than normal multiplication.
- Karatsuba algorithm was invented by Anatoly Karatsuba in 1960.

Problem with Normal Method

Example:

1234 × 5678

Normal method needs **4 multiplications** when we split numbers.

More digits = More calculations = More time

Karatsuba reduces:

4 multiplications

3 multiplications

It uses **Divide and Conquer** method.

Step 1: Split numbers into two parts

Step 2: Multiply smartly

Step 3: Combine the result

Multiply:

1234 × 5678

Split numbers:
1234 → 12 and 34
5678 → 56 and 78

Now calculate:
$z_o = 34 \times 78$
$z_2 = 12 \times 56$
$z_1 = (12+34) \times (56+78)$

Final formula:
Answer = $z_2 \times 10^2 + (z_1 - z_2 - z_o) \times 10 + z_o$
Final Answer = 7006652 ✅

Advantage

Normal Method → $O(n^2)$

Karatsuba → $O(n^{1.585})$

- Less multiplications
- More speed
- Better for large numbers

Application

- Cryptography
- Large number calculations
- Computer algorithms

- Here , $x = 1234$, $y = 8765$, $b = 10$ (Decimal Base System) , $n = 4$ (digits)

We need to write "x" and "y" into 2 halves like this :

$x = 12\ 34$ $\quad\quad$ $y = 87\ 65$

$x_H\ x_L$ $\quad\quad$ $y_H\ y_L$

Now , Steps to follow :

(a) Compute $x_H * y_H$ ⟶ Let say S_1

(b) Compute $x_L * y_L$ ⟶ Let say S_2

(c) Compute $(x_L + x_H) * (y_H + y_L)$ ⟶ Let say S_3

(d) Compute $S_3 - S_2 - S_1$ ⟶ Let say S_4

(e) Compute $S_1 * (b^n) + S_4 * (b^{n/2}) + S_2$ ⟶ Required Result

Now , Here $x_H = 12$, $x_L = 34$, $y_H = 87$, $y_L = 65$, $b = 10$ and $n = 4$

- Step(a) : $S_1 = x_H * y_H = 12 * 87 = 1044$ ⟶ Half Multiplication (R) 1
- Step(b) : $S_2 = x_L * y_L = 34 * 65 = 2210$ ⟶ Half Multiplication (R) 2
- Step(c) : $S_3 = (x_L + x_H) * (y_H + y_L) = (34 + 12) * (87 + 65) = 46 * 162 = 6992$ ⟶ HM (R) 3
- Step(d) : $S_4 = S_3 - S_2 - S_1 = 6992 - 2210 - 1044 = 3738$
- Step(e) : Required Result $= S_1 * (b^n) + S_4 * (b^{n/2}) + S_2$

$= 1044 * (10^4) + 3738 * (10^{4/2}) + 2210$

$= 10440000 + 3738 * 10^2 + 2210$

$= 10440000 + 373800 + 2210$

$= 10,816,010$